# Progress Report Document

**CareFlow Health Management**

**Group Number:** G2

**Group members:** Zakaria Hoque, Tessa Rachelle Esman, Mara Evans, Manuel Gardea, Logan Templin

**Project Progress Report Guidelines**

1. The group leader must rotate weekly.
2. All group members must participate in and approve the progress report. Make this document as a shared document among the group members using Google Doc.
3. The group leader is responsible for updating and submitting the progress report every week. Use this same document for all weekly progress reports.
4. Each week, the group leader and one additional group member will present the progress in a short in-class presentation (5 minutes).
5. In addition to this document, a short video must be submitted weekly on Moodle for grading. You may use this video during the presentation (muted) to demonstrate the running feature, or you may present the implemented feature using a live demo. Any group member could record this video.
6. Support your progress with figures, tables, and screenshots, and reference them in the "What You Did" section. Make sure all figures are clearly labeled in the document.
7. List the tasks assigned to each group member for the week.
8. In the "Update Area," always place the most recent update first.
    - For example, the progress for PR2 should appear before PR1.
    - Each new update should be added directly after the table, above previous entries.

| Report# | Date | Group Leader | What you Did | What you are going to do (next week) |
|---|---|---|---|---|
| PR1 | 2/9/2026 | **Zakaria Hoque** | **Created Basic Wireframes for the FrontEnd, and developed SQLite tables (both can be seen in the screenshots below). Researched python tkinter library for GUI development.** | **Develop GUI using the python tkinter library. Develop basic pages for each subsystem. Create a landing/login page, and connect subsystems to this main page.** |
| PR2 | 2/15/2026 | **Manuel Gardea** | **Implemented records subsystem first 3 use cases / Developing GUI for existing landings** | **Continue developing predefined use cases, finish use cases related to records system, and implementing required GUI** |
| PR3 | 2/22/2026 | **Mara Evans** | **Finished records subsystem / Implemented clinic subsystem / Further work on GUI and connecting subsystems** | **Finish clinic subsystem use cases and further connecting subsystems** |
| PR4 | 3/18/2026 | **Logan Templin** | **Finished the staff management subsystem/further connecting subsystems** | **Continue developing our assigned systems and integrating them together.** |
| PR5 | 3/25/2026 | **Tessa Esman** | **Finished Patient Subsystem/ Refactoring/ Improving GUI design, develop & implement new GUI** | **develop GUI code, ensure design consistency, further refactoring (See notes at bottom of PR5)** |
| PR6 | 4/1/2026 | **Zakaria Hoque** | **Finished Billing Subsystem/ Implementing new GUI** | **Checking GUIs for continuity and connecting all pages through sidebar buttons** |

**Assign Tasks - PR6**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Updated GUI for Records |
| M2 | Tessa R Esman | Designed & Created Dashboard / overhaul Clinic Location GUI |
| M3 | Manuel Gardea | Updated GUI for Staff Management |
| M4 | Logan Templin | Finished up billing system features + Updated GUI |
| M5 | Mara Evans | Updated GUI for Patient Management |

**Assign Tasks - PR5**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Refactored code |
| M2 | Tessa R Esman | Refactored code/ Further GUI development |
| M3 | Manuel Gardea | Refactored code / debugging systems |
| M4 | Logan Templin | Completed download receipt feature/ Worked on a billing system/ Made changes to the record system |
| M5 | Mara Evans | Implemented 3 patient management use cases/ Report |

**Assign Tasks - PR4**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Recorded video/ Helped with developing staff management subsystem |
| M2 | Tessa R Esman | Begin designing the master GUI file layout |
| M3 | Manuel Gardea | Finished implementing staff management subsystem |
| M4 | Logan Templin | Continued working on the billing system and working on the download receipt feature. |
| M5 | Mara Evans | Assisting with GUI design and consistency throughout |

**Assign Tasks - PR3**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Finished the search use case for the records system |
| M2 | Tessa R Esman | Started implementing Location subsystem - first 3 use cases |
| M3 | Manuel Gardea | Working on Staff Management subsystem |
| M4 | Logan Templin | Updating landing page and connecting subsystems / Working on Billing subsystem + added- add new staff feature. |
| M5 | Mara Evans | Report / Working on Patient subsystem |

**Assign Tasks – PR2**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Upload, Download, Delete use cases for the record system |
| M2 | Tessa R Esman | More wireframes / Interface dev / Use case frontend / Subsystem plan |
| M3 | Manuel Gardea | Report / Staff Management Backend / Use case Diagram |
| M4 | Logan Templin | Created landing page / updated database, Added feature add new patient |
| M5 | Mara Evans | Subsystem planning and assisted GUI design |

**Assign Tasks – PR1**

| Who Did What | | |
|---|---|---|
| | **Name** | **Role - Completed Tasks** |
| M1 | Zakaria Hoque | Delegated Tasks / Research / Report |
| M2 | Tessa R Esman | Logo Ideation / Frontend Interface Development / Wireframe Part1 |
| M3 | Manuel Gardea | Slides / Research /SQLite Tables |
| M4 | Logan Templin | SQLite Tables |
| M5 | Mara Evans | Research / GUI Development |

**Update Area**

Support your progress report with figures and diagrams, which you may include in this section.

**PR6: Completed Features for Billing Subsystem**

**PR6: Billing Subsystem**

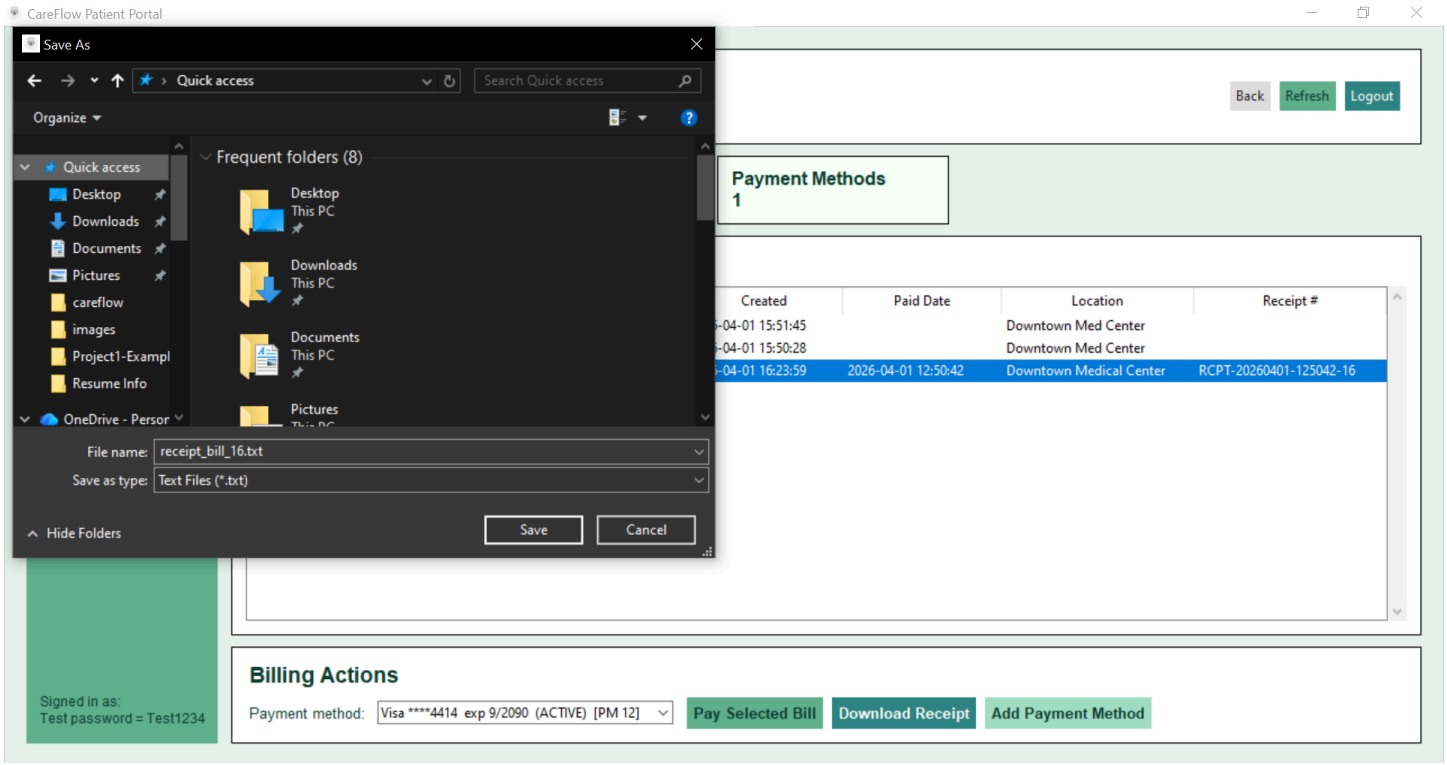
**Use Case 1: Download Receipt**

**Description: The Download Receipt feature uses a SELECT query to retrieve all information needed to build the receipt file. It gathers data from the Bill table, joins the Patient table to get the patient's name and email, joins the ClinicLocation table to get the clinic name, and joins the PaymentMethod table to show the payment type and last four digits used for payment.**

**Process:**

1. **User Logs in to patient billing page**
2. **User selects a bill to pay**
3. **Once bill is paid, hit download receipt**
4. **File window opens to save receipt to your system.**
5. **A textbox appears to confirm the change, click "OK"**

**User case Screenshot:**




Receipt View:

receipt_bill_16.txt - Notepad

File Edit Format View Help

```
CareFlow Payment Receipt
========================
Receipt Number: RCPT-20260401-125042-16
Bill ID: 16
Patient: Test password = Test1234
Email: test@email.com
Clinic: Downtown Medical Center
Amount Paid: $40.00
Due Date: 2026-05-09
Created At: 2026-04-01 16:23:59
Paid Date: 2026-04-01 12:50:42
Payment Method: Visa ****4414
Status: paid
```

**Database: (Receipt numbers are logged once the bill is paid).**

Table: Bill

Filter in any column

| | bill_id | patient_id | location_id | amount | due_date | status | created_at | paid_date | payment_method_id | receipt |
|---|---|---|---|---|---|---|---|---|---|---|
| | Filter | Filter | Filter | Filter | Filter | Filter | Filter | Filter | Filter | Filter |
| 1 | 1 | 1 | 1 | 120.5 | 2026-03-01 | unpaid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 2 | 2 | 2 | 2 | 85.0 | 2026-03-05 | paid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 3 | 3 | 3 | 3 | 200.75 | 2026-03-10 | unpaid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 4 | 4 | 4 | 4 | 45.25 | 2026-03-12 | paid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 5 | 5 | 5 | 5 | 310.0 | 2026-03-15 | unpaid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 6 | 6 | 6 | 6 | 95.6 | 2026-03-18 | paid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 7 | 7 | 7 | 7 | 150.0 | 2026-03-20 | paid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 8 | 8 | 8 | 8 | 60.4 | 2026-03-22 | unpaid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 9 | 9 | 9 | 9 | 500.0 | 2026-03-25 | unpaid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 10 | 10 | 10 | 10 | 75.99 | 2026-03-28 | paid | 2026-02-11 17:09:30 | NULL | NULL | NULL |
| 11 | 11 | 5 | 5 | 40.0 | 2026-03-11 | paid | 2026-03-03 17:58:57 | NULL | NULL | NULL |
| 12 | 12 | 9 | 1 | 75.0 | 2028-01-14 | unpaid | 2026-03-09 02:45:36 | NULL | NULL | NULL |
| 13 | 13 | 15 | 10 | 745.0 | 2026-04-20 | paid | 2026-03-23 17:48:59 | 2026-03-23 14:12:15 | 11 | RCPT-202603 |
| 14 | 14 | 20 | 10 | 120.0 | 2026-07-22 | paid | 2026-04-01 15:50:28 | 2026-04-01 13:24:11 | 12 | RCPT-202604 |
| 15 | 15 | 20 | 10 | 50.0 | 2026-07-22 | unpaid | 2026-04-01 15:51:45 | NULL | NULL | NULL |
| 16 | 16 | 20 | 17 | 40.0 | 2026-05-09 | paid | 2026-04-01 16:23:59 | 2026-04-01 12:50:42 | 12 | RCPT-202604 |
| 17 | 17 | 20 | 22 | 43.0 | 2027-09-03 | unpaid | 2026-04-01 17:26:25 | NULL | NULL | NULL |

**Queries: (pull info to create the receipt)**

```
rows = self._db().cursor().execute("""
    SELECT
        b.bill_id,
        b.amount,
        b.due_date,
        b.status,
        b.created_at,
        b.paid_date,
        COALESCE(cl.name, '') AS location_name,
        COALESCE(b.receipt_number, '')
    FROM Bill b
    LEFT JOIN ClinicLocation cl ON cl.location_id = b.location_id
    WHERE b.patient_id = ?
    ORDER BY
        CASE WHEN b.status = 'paid' THEN 1 ELSE 0 END,
        b.due_date ASC,
        b.created_at DESC
""", (self.logged_in_patient_id,)).fetchall()
```

```
self._db().execute("""
    UPDATE Bill
    SET status = 'paid',
        paid_date = ?,
        payment_method_id = ?,
        receipt_number = ?
    WHERE bill_id = ?
""", (paid_date, payment_method_id, receipt_number, bill_id))
self._db().commit()
```

**PR6: Completed Features for Billing Subsystem**

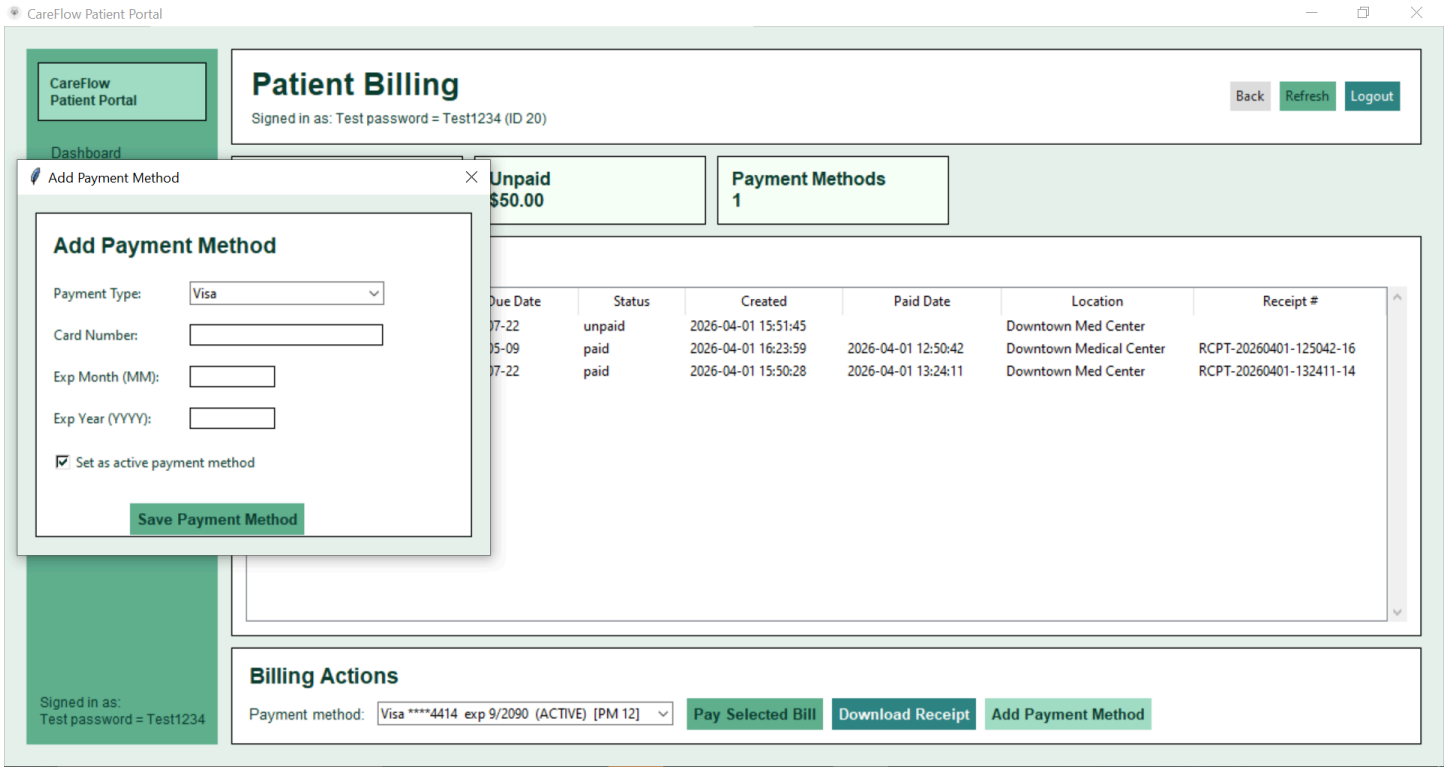
**PR6: Billing Subsystem**

**Use Case 1: Allows a patient to add a new payment method to their account for billing purposes. Using the insert into command to add it to the database.**

**Process:**

6. **User hits “add payment method”**
7. **Patient enters card type, card number, and expiration date**
8. **System validates the input**
9. **Existing active payment methods for that patient are deactivated**
10. **The new payment method is inserted into the database**
11. **Only the last four digits of the card are stored for security**

**User case Screenshot:**




**Database:**

| | payment_method_id | patient_id | type | last4 | exp_month | exp_year | billing_zip | active_flag |
|---|---|---|---|---|---|---|---|---|
| | Filter | Filter | Filter | Filter | Filter | Filter | Filter | Filter |
| 1 | 1 | 1 | card | 1234 | 5 | 2027 | 48201 | 1 |
| 2 | 2 | 2 | card | 2345 | 6 | 2026 | 48202 | 1 |
| 3 | 3 | 3 | bank | 3456 | 7 | 2028 | 48203 | 1 |
| 4 | 4 | 4 | card | 4567 | 8 | 2027 | 48204 | 1 |
| 5 | 5 | 5 | card | 5678 | 9 | 2029 | 48205 | 1 |
| 6 | 6 | 6 | bank | 6789 | 10 | 2026 | 48206 | 1 |
| 7 | 7 | 7 | card | 7890 | 11 | 2027 | 48207 | 1 |
| 8 | 8 | 8 | card | 8901 | 12 | 2028 | 48226 | 1 |
| 9 | 9 | 9 | bank | 9012 | 1 | 2029 | 48210 | 1 |
| 10 | 10 | 10 | card | 0123 | 2 | 2027 | 48211 | 1 |

**Queries:**

```
if active_flag == 1:
    cur.execute("""
        UPDATE PaymentMethod
        SET active_flag = 0
        WHERE patient_id = ?
    """, (self.logged_in_patient_id,))

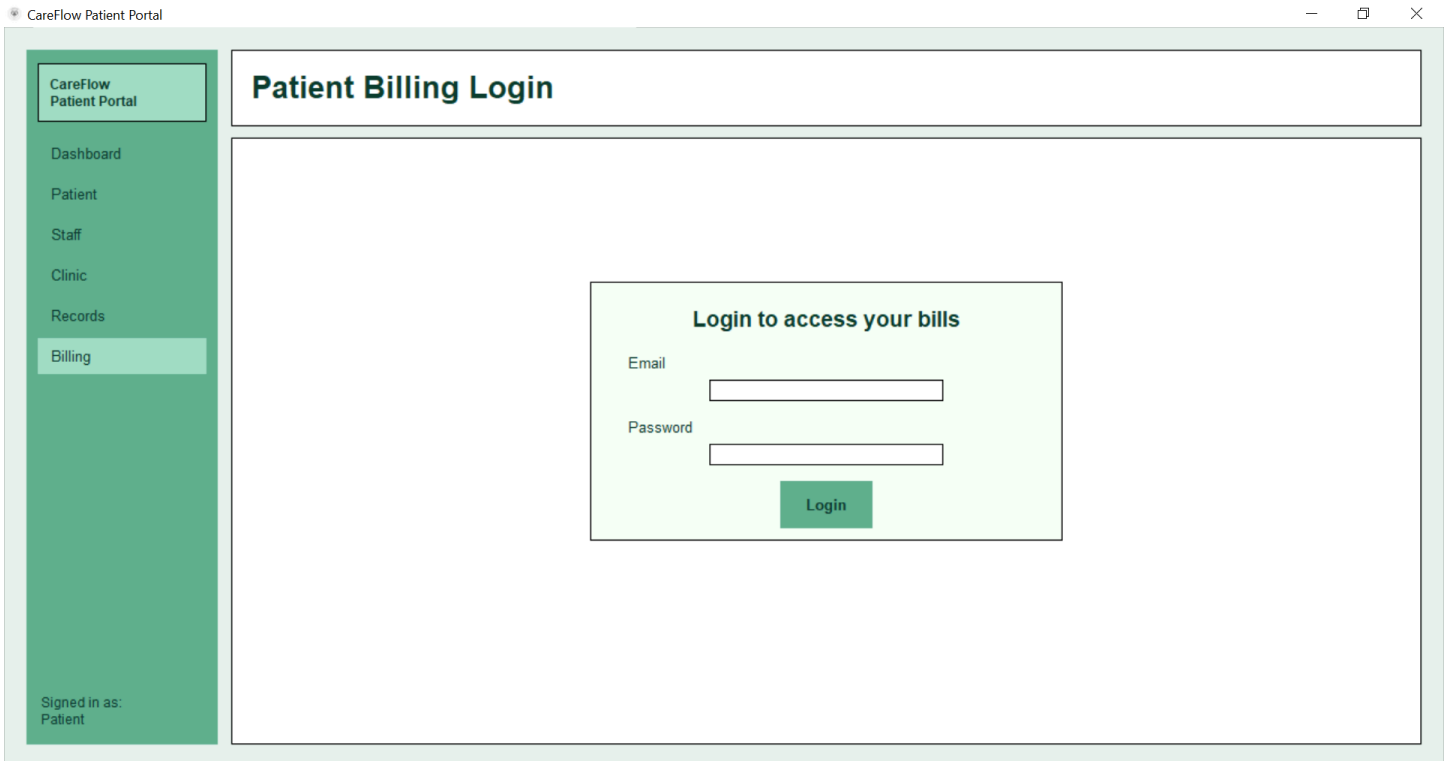
cur.execute("""
    INSERT INTO PaymentMethod (
        patient_id, type, last4, exp_month, exp_year, active_flag
    ) VALUES (?, ?, ?, ?, ?, ?)
""", (
    self.logged_in_patient_id,
    pm_type,
    last4,
    exp_month_db,
    exp_year_db,
    active_flag
))
```

Restart Visual Studio

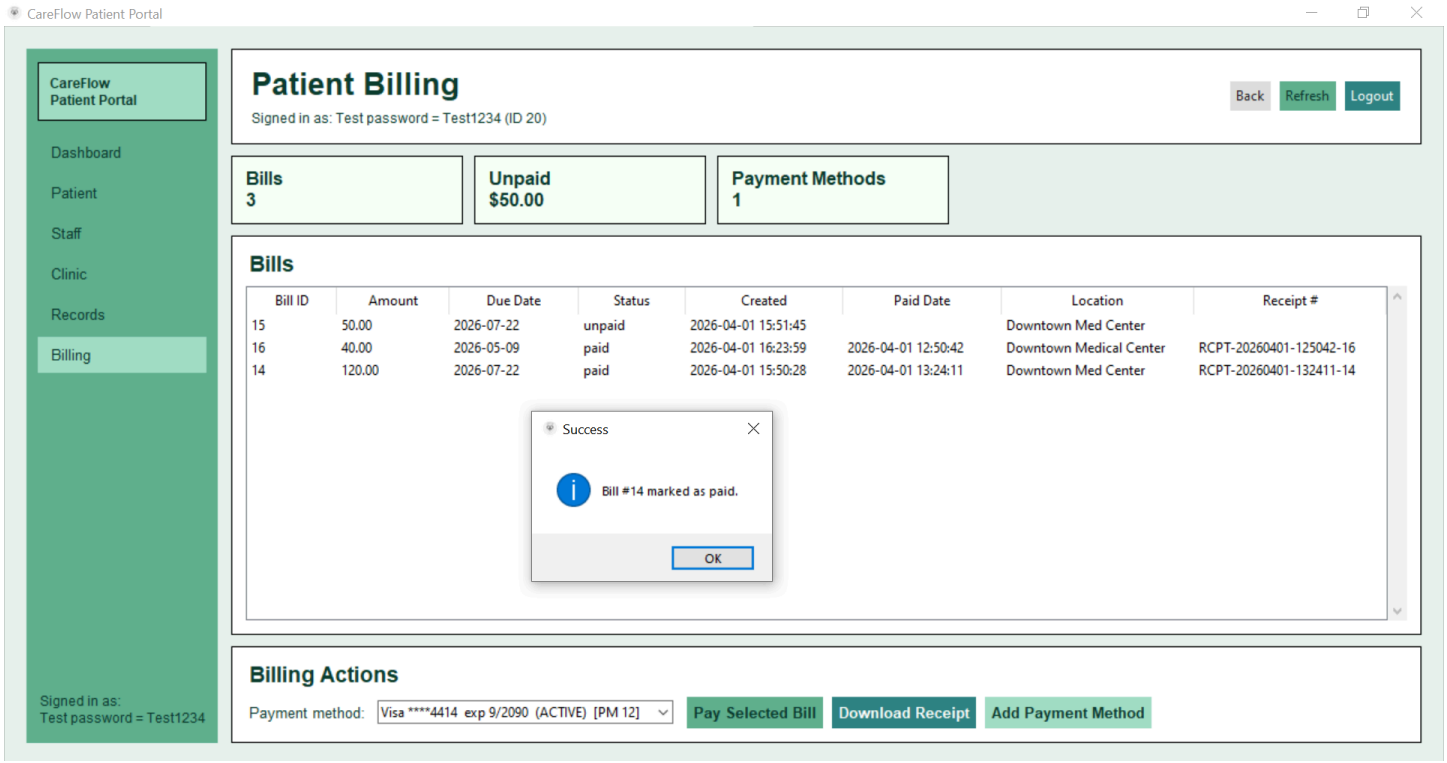
# Work in progress final styling of pages (GUI):

## Billing:

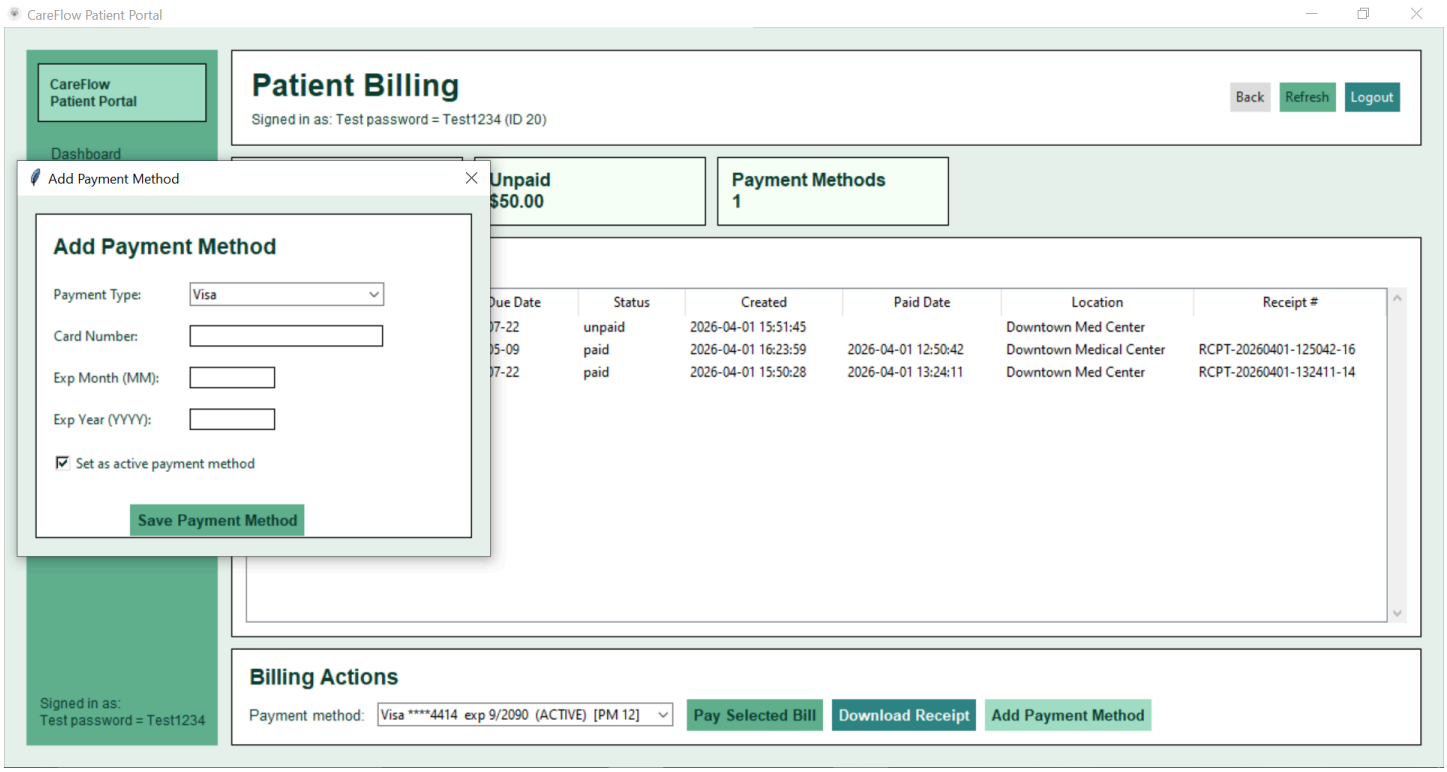
**Patient Billing login:**



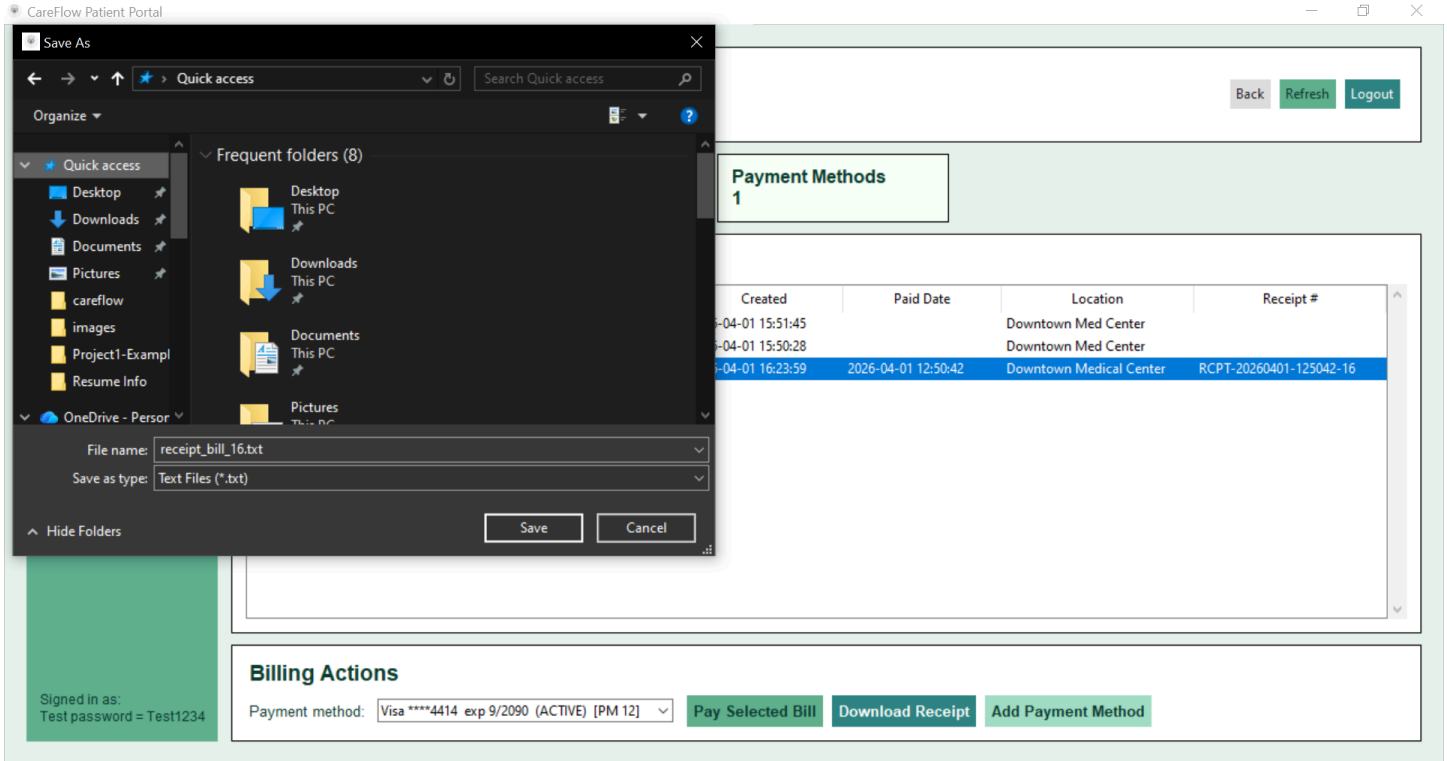

**Patient Billing Pay Bill:**



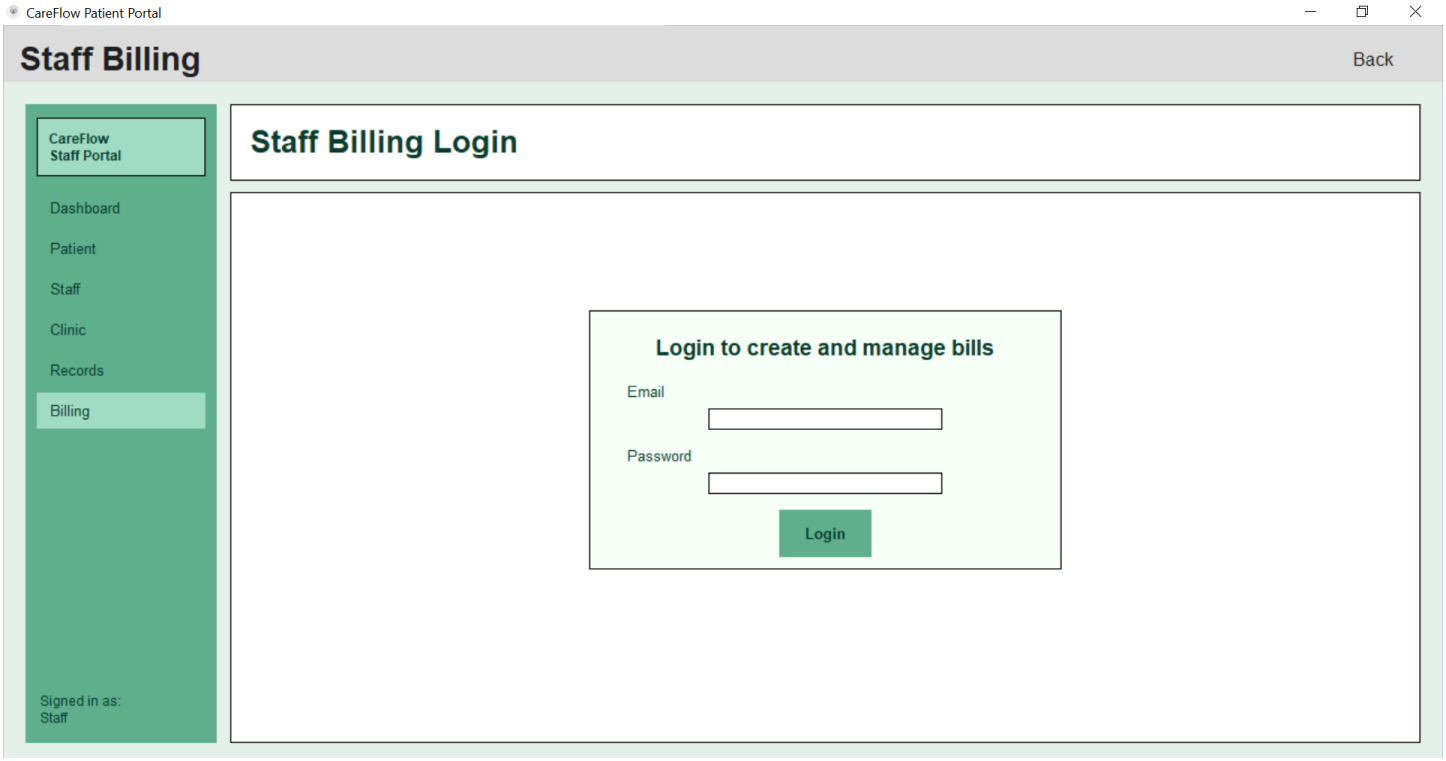
**Patient Billing add a payment method:**



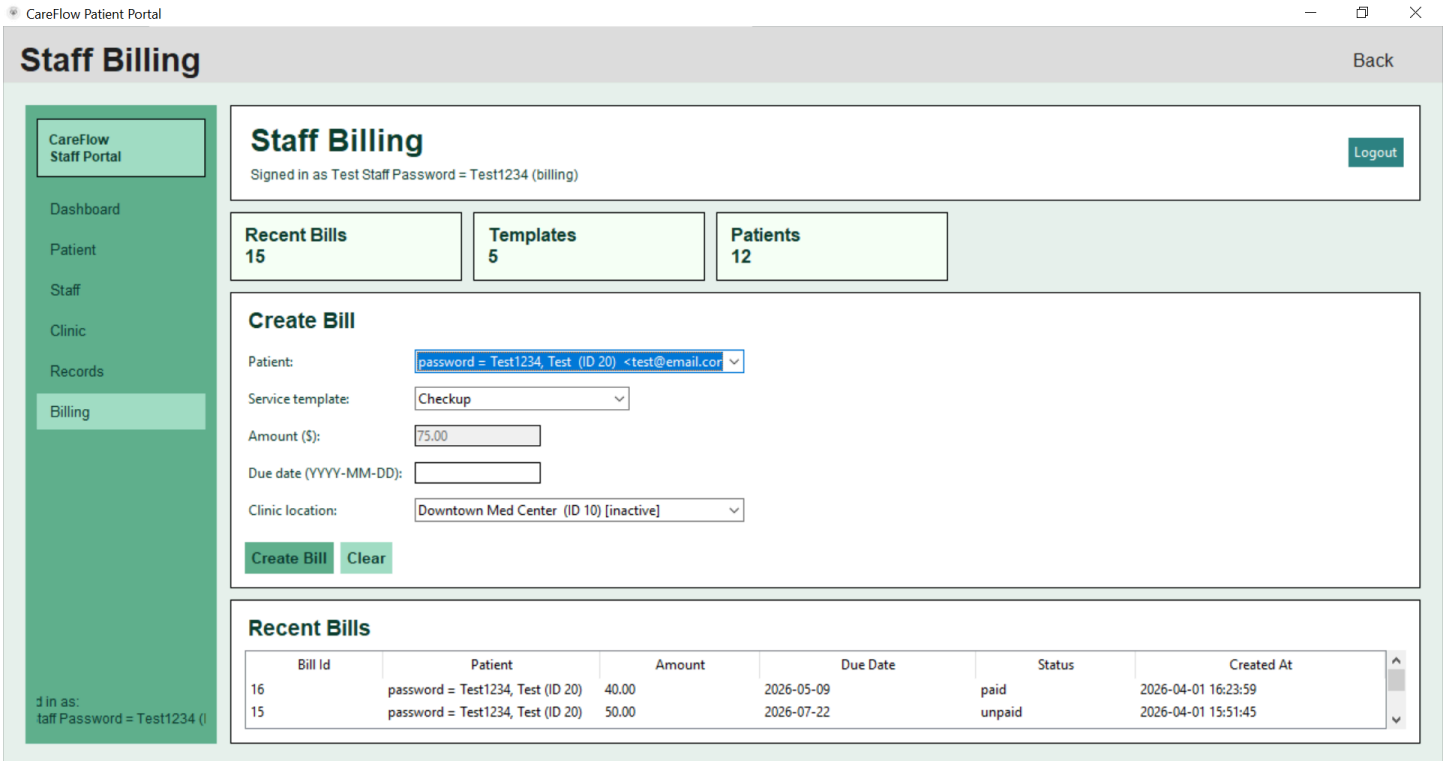

**Patient Billing download a receipt:**



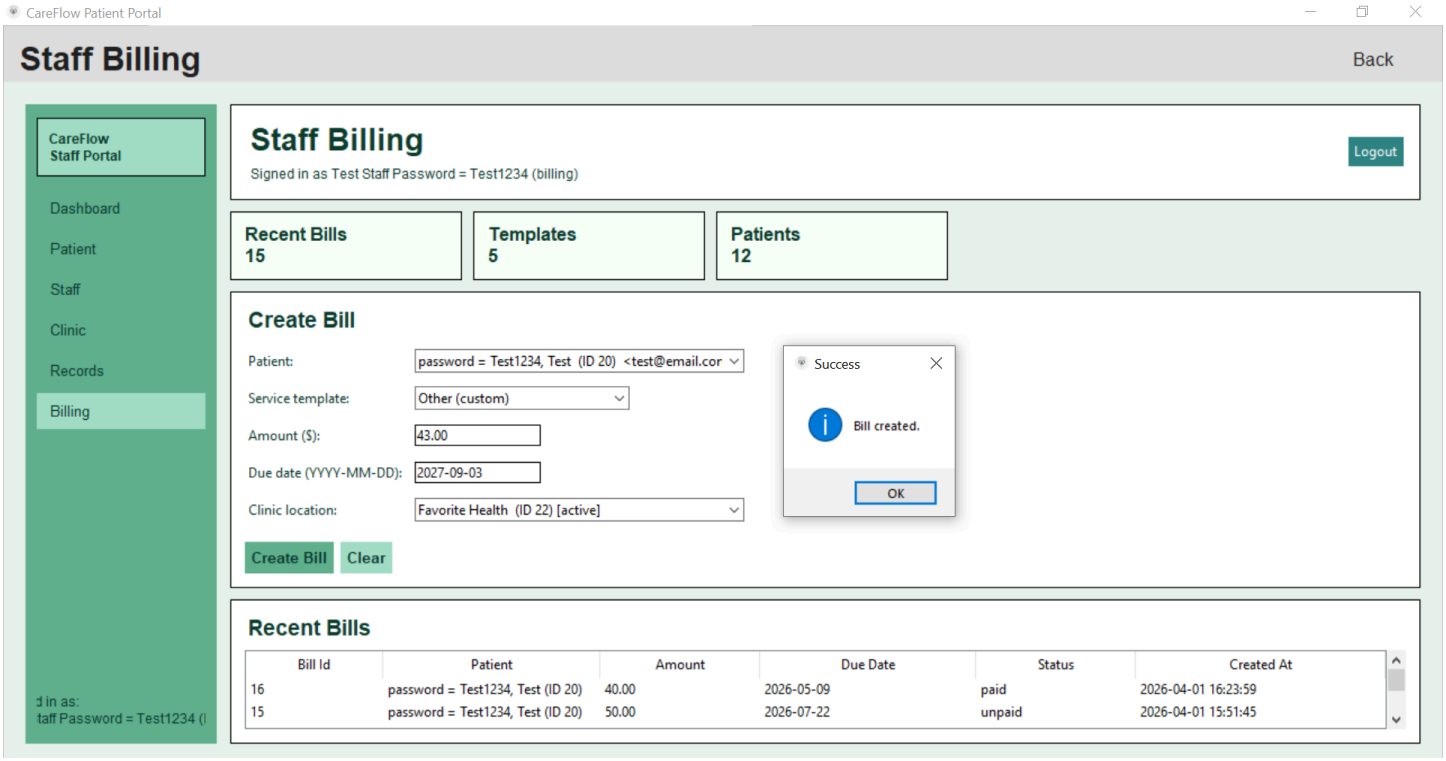
**Staff Billing login:**



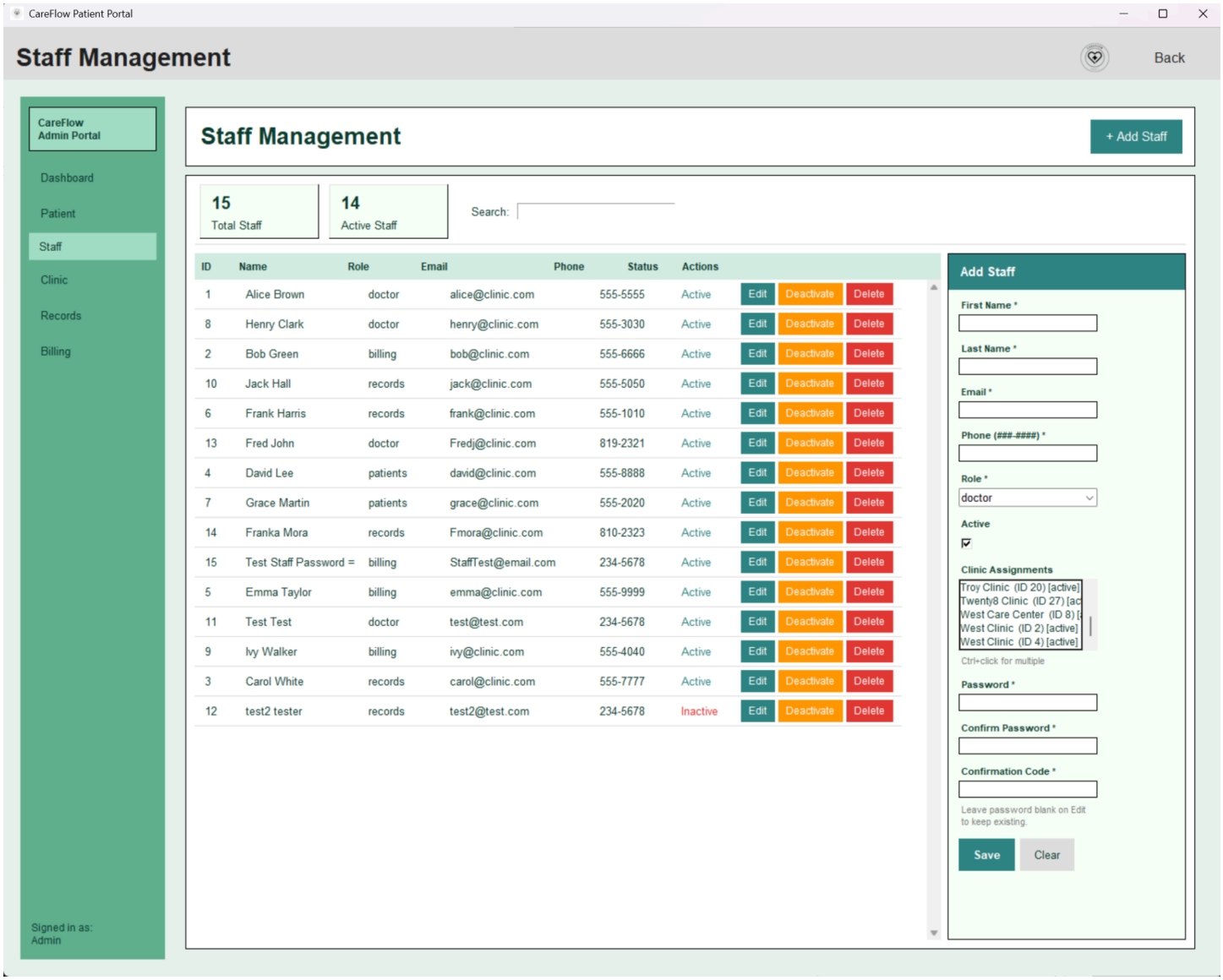

**Staff Billing main page:**



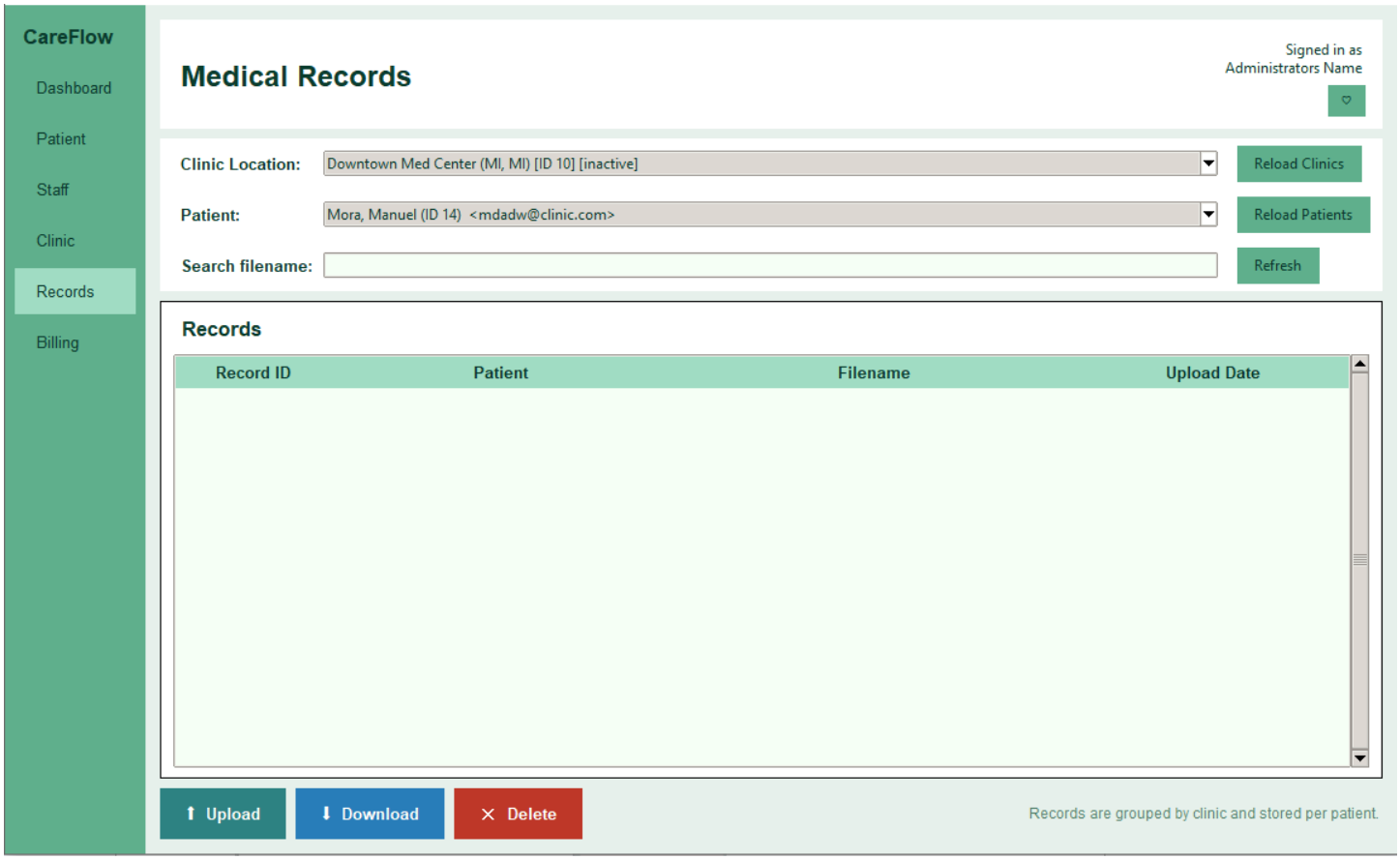
**Staff Billing create a bill:**

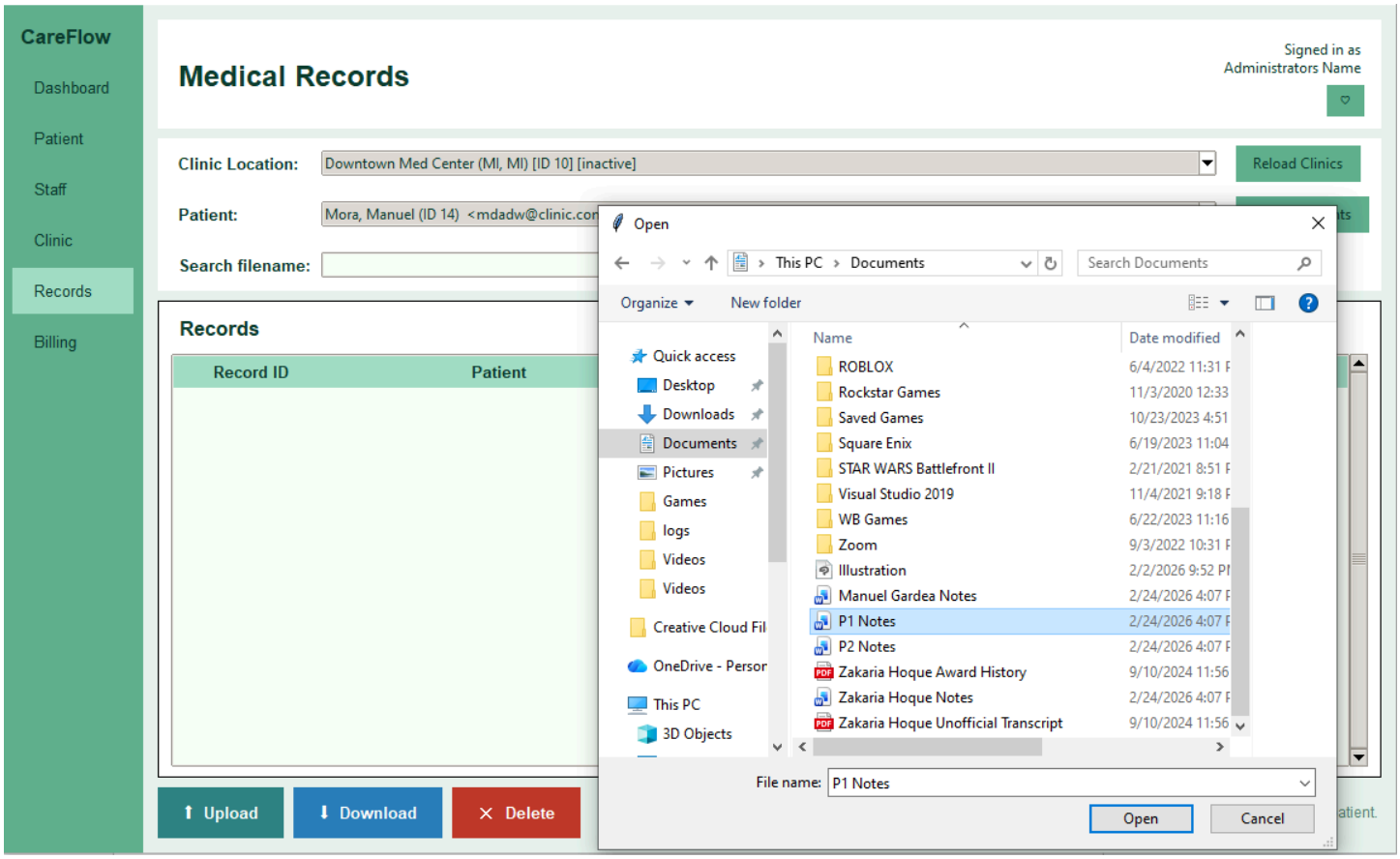


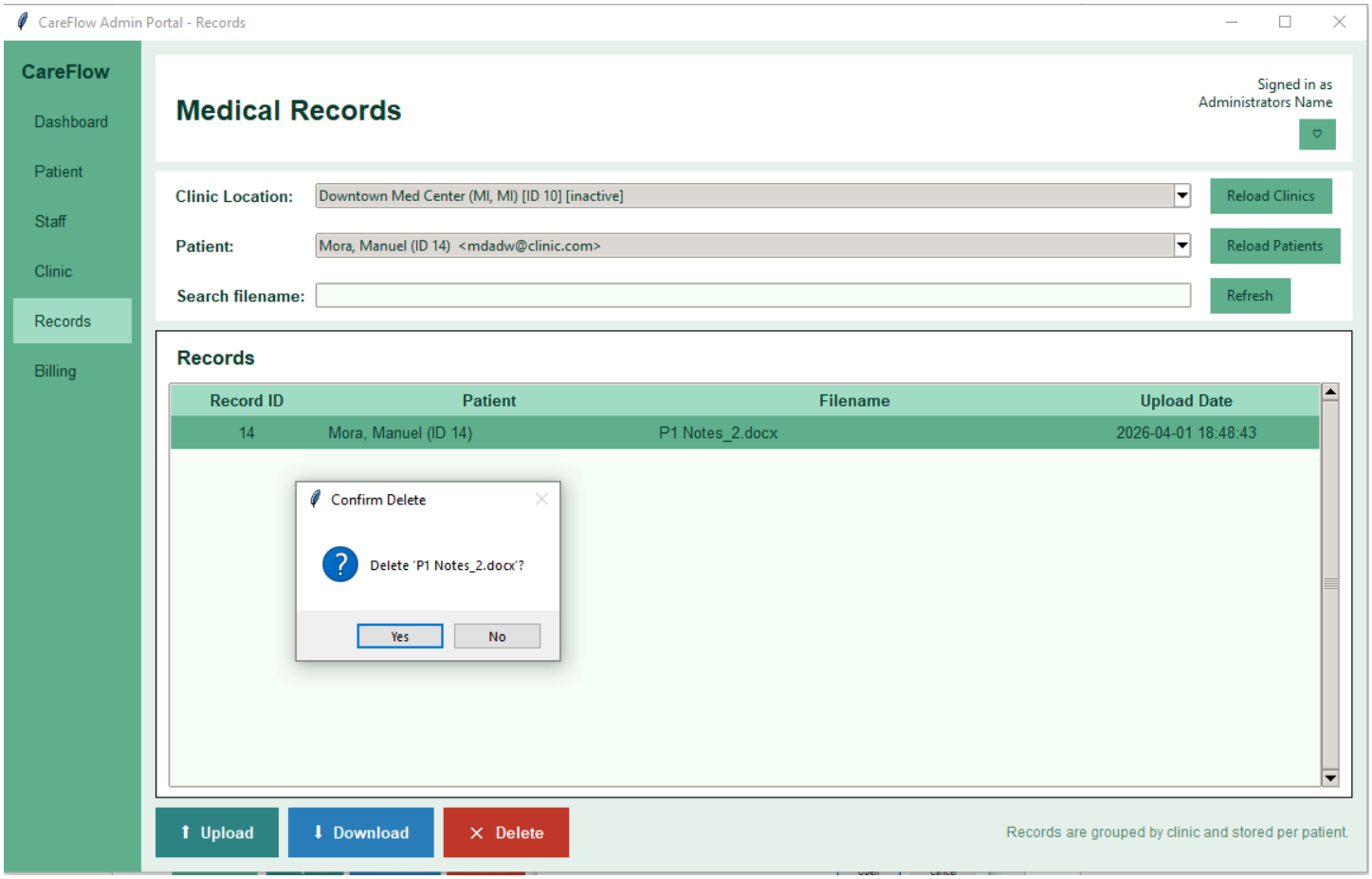

# Staff Management:


CareFlow Patient Portal
Staff Management
Back
CareFlow Admin Portal
Dashboard
Patient
Staff
Clinic
Records
Billing
Signed in as: Admin
Staff Management
+ Add Staff
15
Total Staff
14
Active Staff
Search:
ID Name Role Email Phone Status Actions
1 Alice Brown doctor alice@clinic.com 555-5555 Active Edit Deactivate Delete
8 Henry Clark doctor henry@clinic.com 555-3030 Active Edit Deactivate Delete
2 Bob Green billing bob@clinic.com 555-6666 Active Edit Deactivate Delete
10 Jack Hall records jack@clinic.com 555-5050 Active Edit Deactivate Delete
6 Frank Harris records frank@clinic.com 555-1010 Active Edit Deactivate Delete
13 Fred John doctor Fredj@clinic.com 819-2321 Active Edit Deactivate Delete
4 David Lee patients david@clinic.com 555-8888 Active Edit Deactivate Delete
7 Grace Martin patients grace@clinic.com 555-2020 Active Edit Deactivate Delete
14 Franka Mora records Fmora@clinic.com 810-2323 Active Edit Deactivate Delete
15 Test Staff Password = billing StaffTest@email.com 234-5678 Active Edit Deactivate Delete
5 Emma Taylor billing emma@clinic.com 555-9999 Active Edit Deactivate Delete
11 Test Test doctor test@test.com 234-5678 Active Edit Deactivate Delete
9 Ivy Walker billing ivy@clinic.com 555-4040 Active Edit Deactivate Delete
3 Carol White records carol@clinic.com 555-7777 Active Edit Deactivate Delete
12 test2 tester records test2@test.com 234-5678 Inactive Edit Deactivate Delete
Add Staff
First Name *
Last Name *
Email *
Phone (###-####) *
Role *
doctor
Active
Clinic Assignments
Troy Clinic (ID 20) [active]
Twenty8 Clinic (ID 27) [ac
West Care Center (ID 8) [
West Clinic (ID 2) [active]
West Clinic (ID 4) [active]
Ctrl+click for multiple
Password *
Confirm Password *
Confirmation Code *
Leave password blank on Edit to keep existing.
Save
Clear

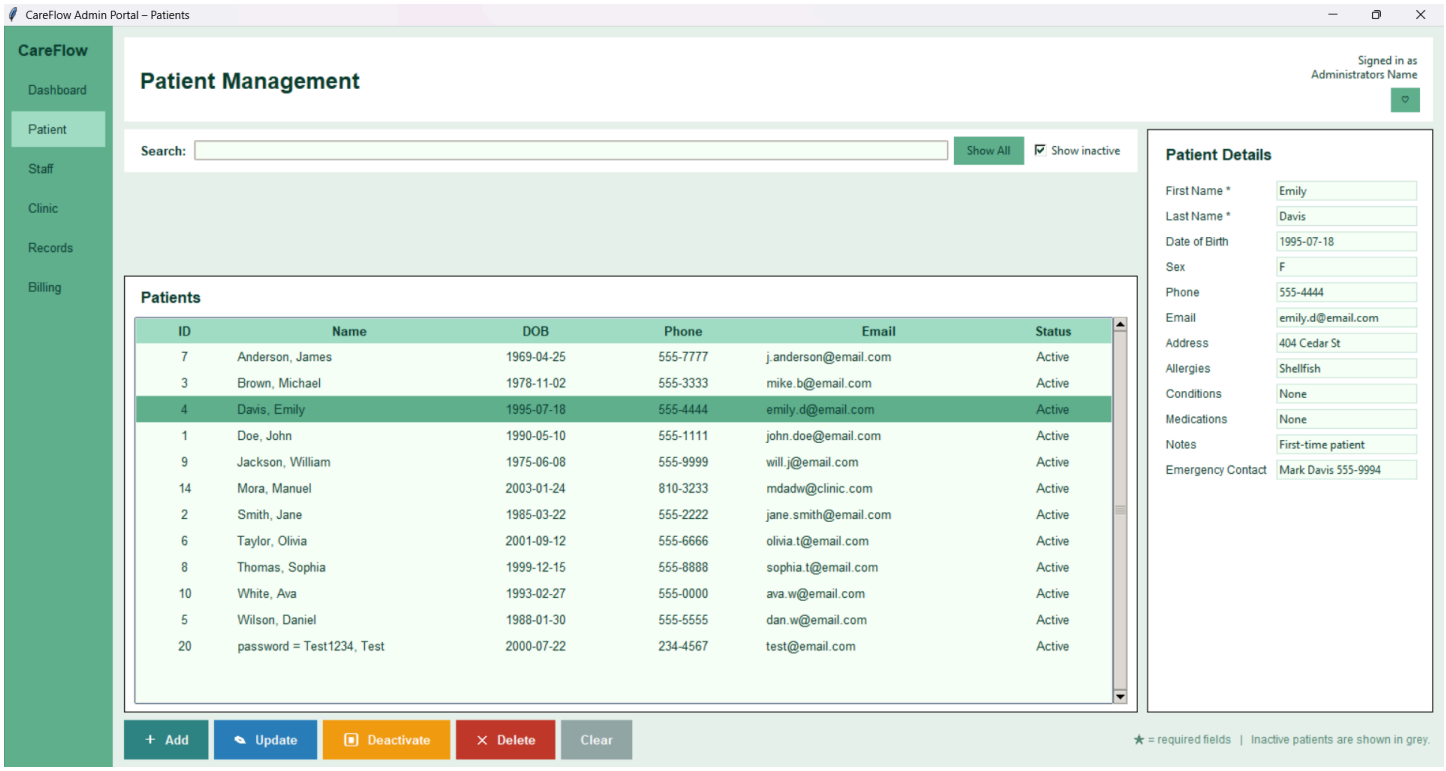
# Records

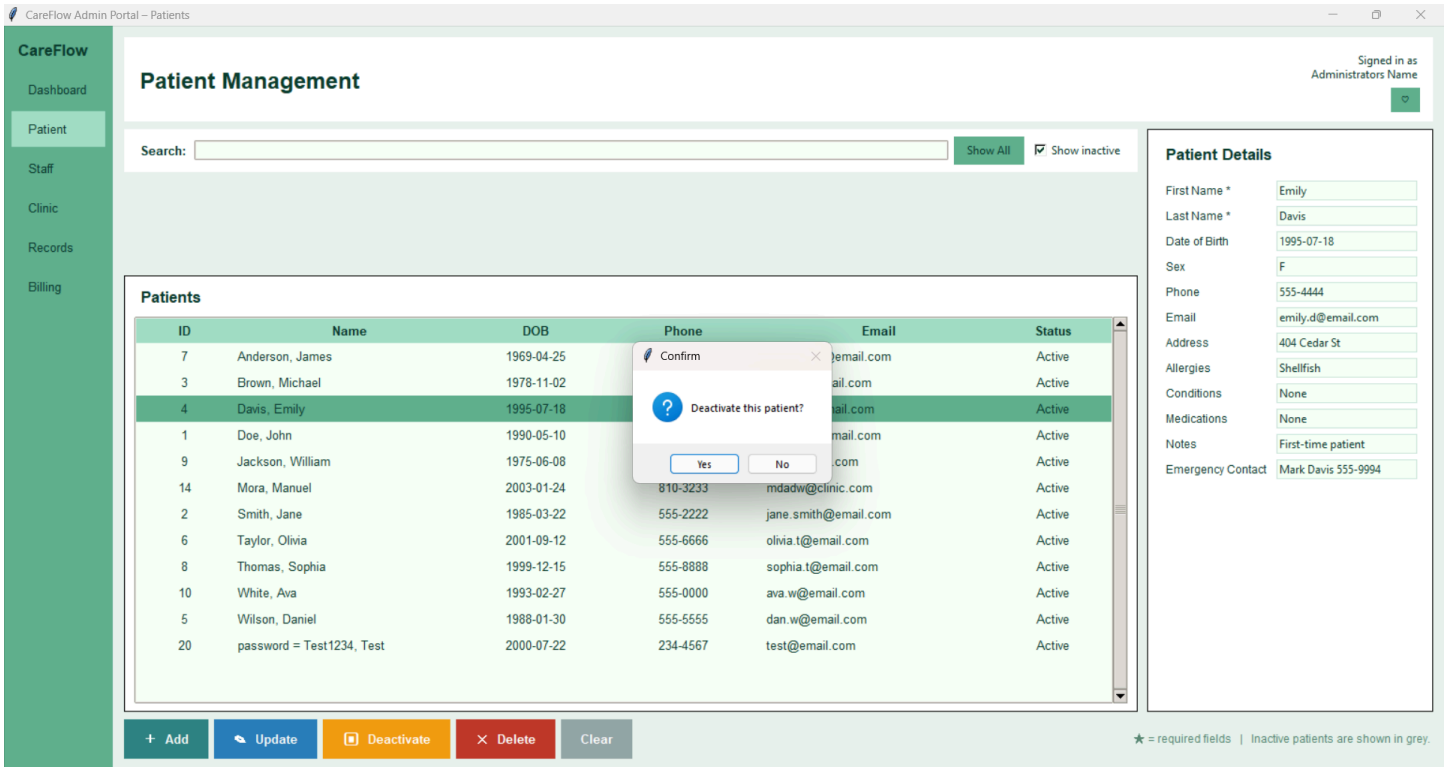

CareFlow
Dashboard
Patient
Staff
Clinic
Records
Billing
Medical Records
Signed in as
Administrators Name
Clinic Location:
Downtown Med Center (MI, MI) [ID 10] [inactive]
Reload Clinics
Patient:
Mora, Manuel (ID 14) <mdadw@clinic.com>
Reload Patients
Search filename:
Refresh
Records
Record ID
Patient
Filename
Upload Date
Upload
Download
Delete
Records are grouped by clinic and stored per patient.

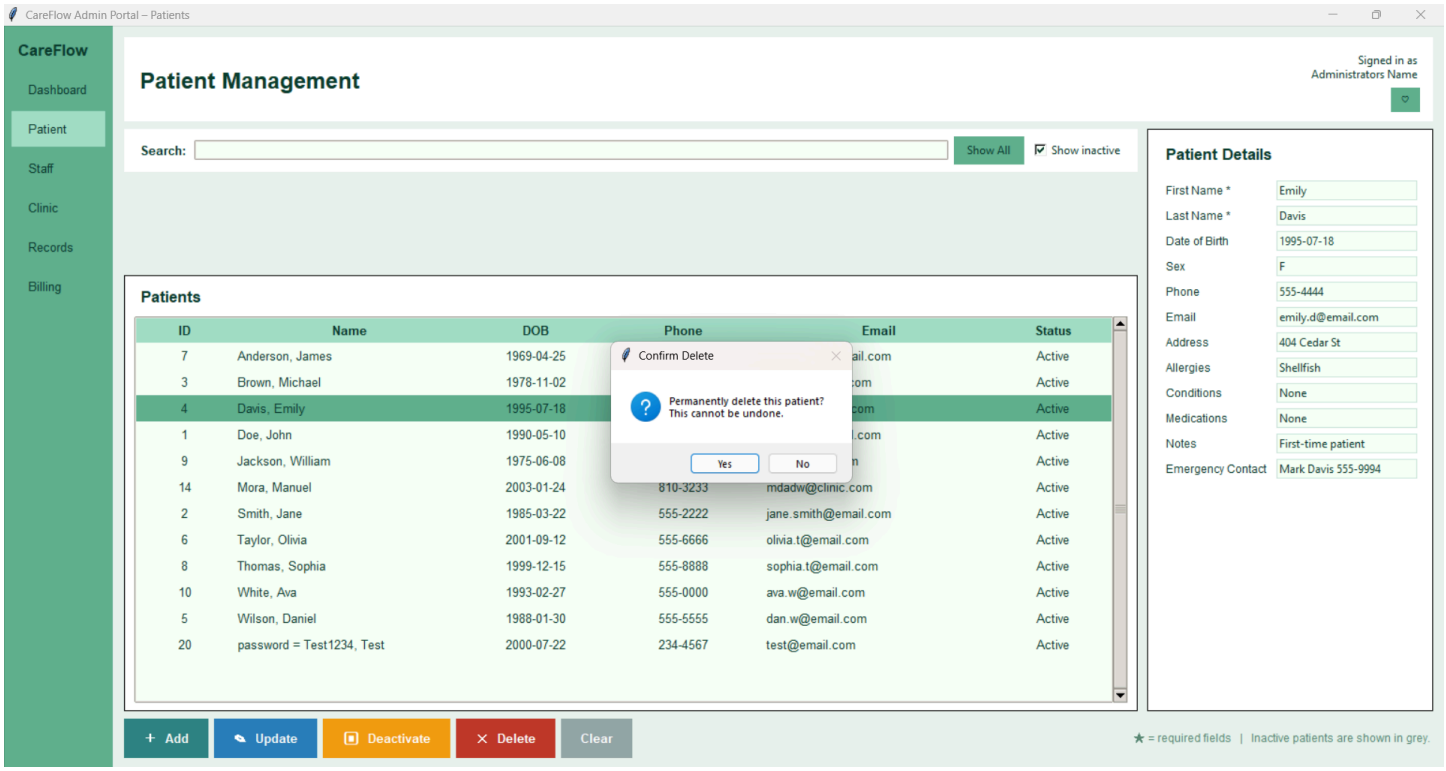

CareFlow
Dashboard
Patient
Staff
Clinic
Records
Billing
Medical Records
Signed in as
Administrators Name
Clinic Location:
Downtown Med Center (MI, MI) [ID 10] [inactive]
Reload Clinics
Patient:
Mora, Manuel (ID 14) <mdadw@clinic.com
Search filename:
Records
Record ID
Patient
Upload
Download
Delete
Open
This PC › Documents
Search Documents
Organize
New folder
Quick access
Desktop
Downloads
Documents
Pictures
Games
logs
Videos
Videos
Creative Cloud Fil
OneDrive - Persor
This PC
3D Objects
Name
Date modified
ROBLOX
Rockstar Games
Saved Games
Square Enix
STAR WARS Battlefront II
Visual Studio 2019
WB Games
Zoom
Illustration
Manuel Gardea Notes
P1 Notes
P2 Notes
Zakaria Hoque Award History
Zakaria Hoque Notes
Zakaria Hoque Unofficial Transcript
File name:
P1 Notes
Open
Cancel


CareFlow Admin Portal - Records
CareFlow
Dashboard
Patient
Staff
Clinic
Records
Billing
Medical Records
Signed in as
Administrators Name
Clinic Location:
Downtown Med Center (MI, MI) [ID 10] [inactive]
Reload Clinics
Patient:
Mora, Manuel (ID 14) <mdadw@clinic.com>
Reload Patients
Search filename:
Refresh
Records
Record ID
Patient
Filename
Upload Date
14
Mora, Manuel (ID 14)
P1 Notes_2.docx
2026-04-01 18:48:43
Confirm Delete
Delete 'P1 Notes_2.docx'?
Yes
No
Upload
Download
Delete
Records are grouped by clinic and stored per patient.

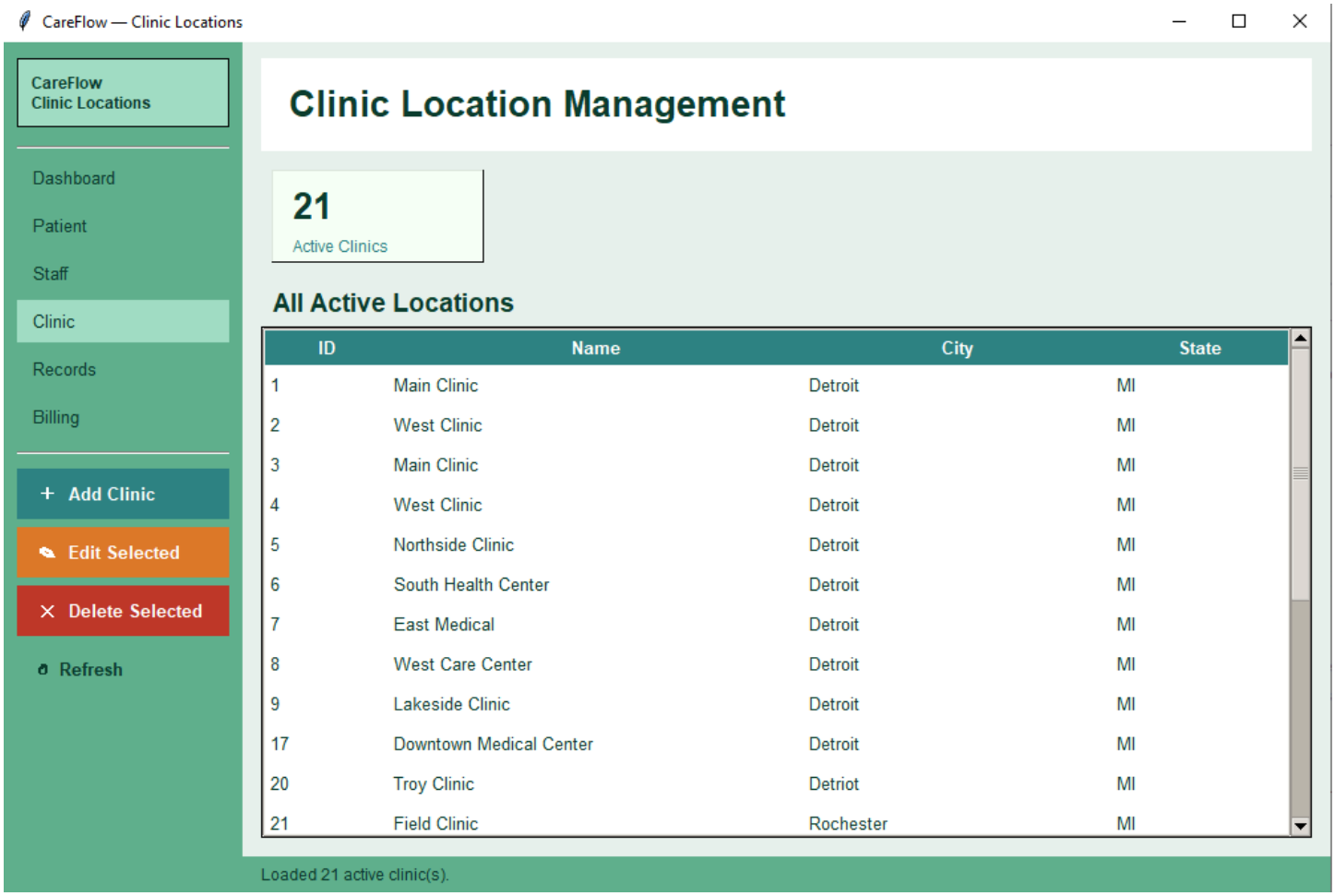
# Patient Management

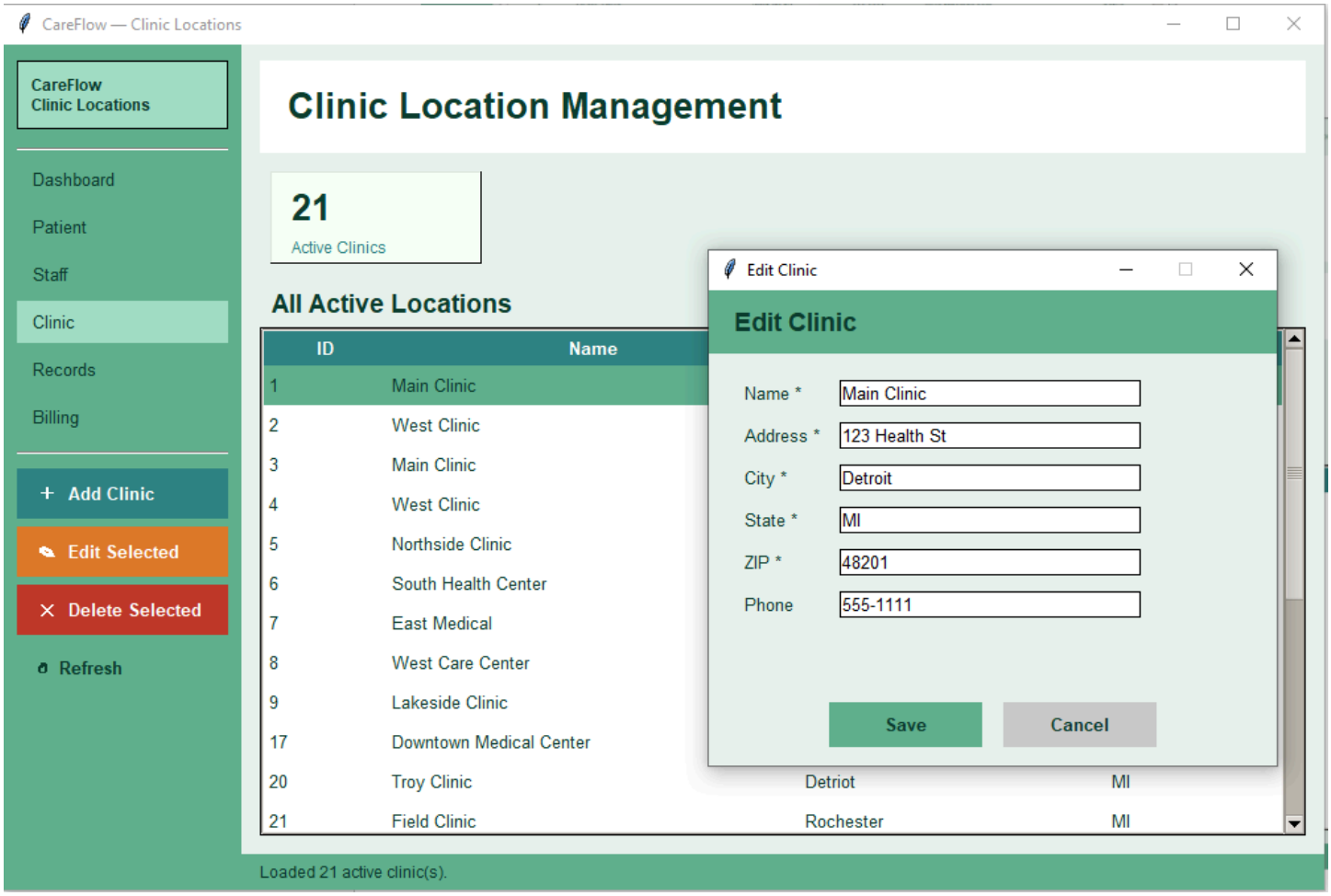





# Clinic Location Management



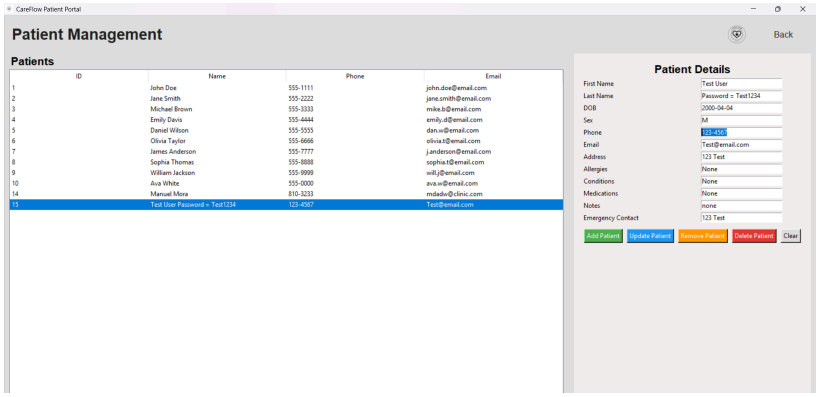



**PR6: Complete**

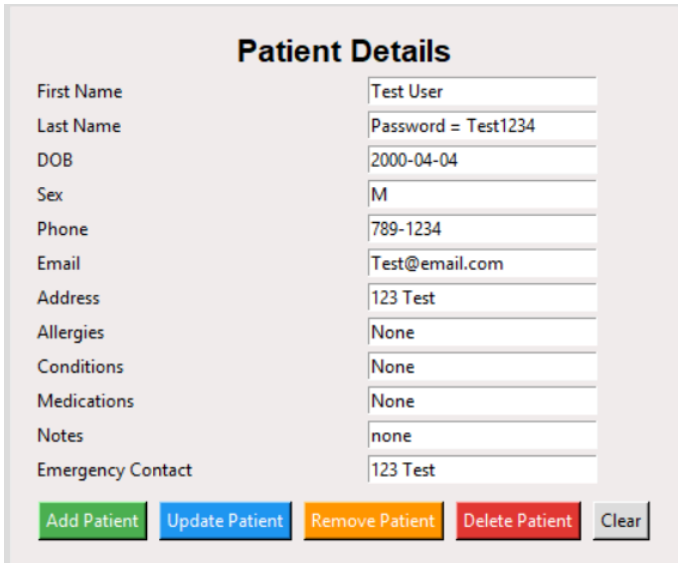
**PR5: Completed Features for Patient Subsystem**
**PR5: Patient Subsystem**
**Use Case 1: Update Patient Information**
**Description: User can edit a selected patient's information in form field**
**Process:**

12. **Select an existing patient from the list by clicking on them**
13. **On the right are the input fields that fill with data, click on a field and type to edit**
14. **After updating information, click the blue "Update Patient" button**
15. **A textbox appears to confirm the change, click "OK"**

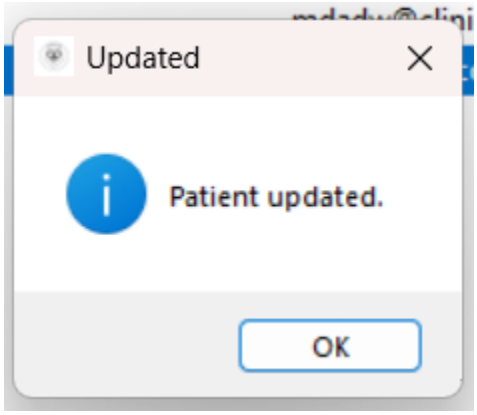
**User case Screenshots:**




**Choose a user and field to edit**




**Edit the field (in this case Phone) then click Update Patient**




**A popup will confirm that the patient information was update**

| 15 | Test User Password = Test1234 | 789-1234 | Test@email.com |
|---|---|---|---|

**The table will update with the new information**

**Database:**

| | patient_id INTEGER PRIMARY KEY AUTOINCREMENT | first_name TEXT | last_name TEXT | dob TEXT | sex TEXT | phone TEXT | email TEXT |
|---|---|---|---|---|---|---|---|
| 1 | 1 | John | Doe | 1990-05-10 | M | 555-1111 | john.doe@email.com |
| 2 | 2 | Jane | Smith | 1985-03-22 | F | 555-2222 | jane.smith@email.com |
| 3 | 3 | Michael | Brown | 1978-11-02 | M | 555-3333 | mike.b@email.com |
| 4 | 4 | Emily | Davis | 1995-07-18 | F | 555-4444 | emily.d@email.com |
| 5 | 5 | Daniel | Wilson | 1988-01-30 | M | 555-5555 | dan.w@email.com |
| 6 | 6 | Olivia | Taylor | 2001-09-12 | F | 555-6666 | olivia.t@email.com |
| 7 | 7 | James | Anderson | 1969-04-25 | M | 555-7777 | j.anderson@email.com |
| 8 | 8 | Sophia | Thomas | 1999-12-15 | F | 555-8888 | sophia.t@email.com |
| 9 | 9 | William | Jackson | 1975-06-08 | M | 555-9999 | will.j@email.com |
| 10 | 10 | Ava | White | 1993-02-27 | F | 555-0000 | ava.w@email.com |
| 11 | 14 | Manuel | Mora | 2003-01-24 | M | 810-3233 | mdadw@clinic.com |
| 12 | 15 | Test User | Password = Test1234 | 2000-04-04 | M | 789-1234 | Test@email.com |

**Queries:**

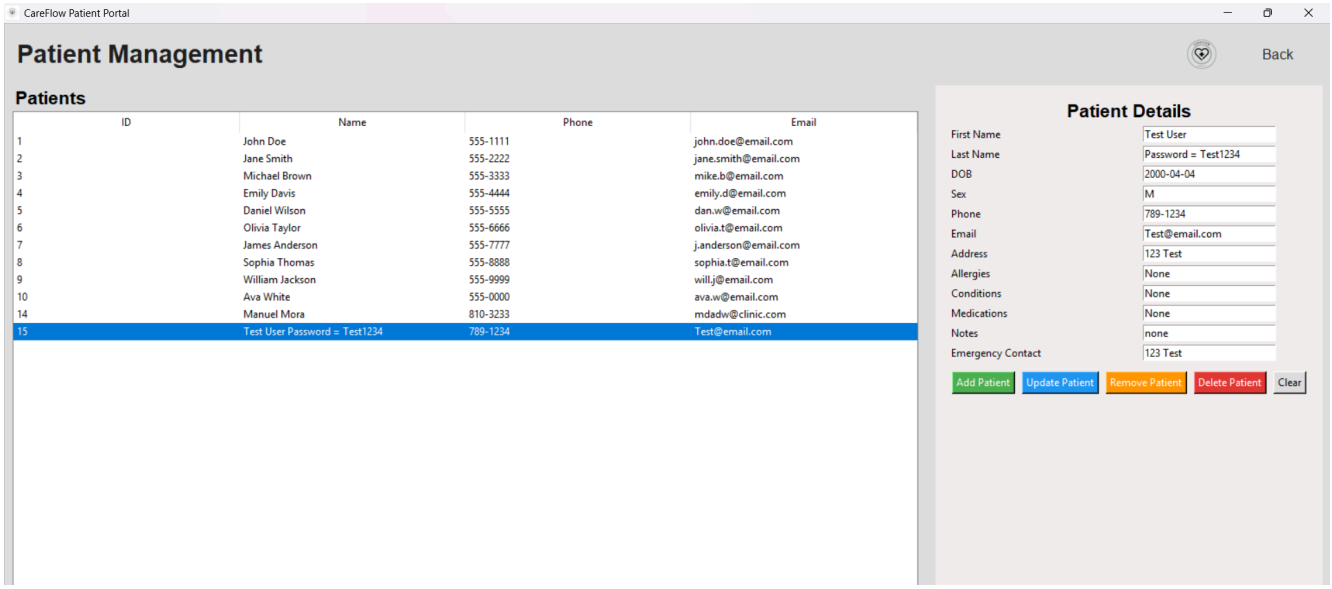
```
conn = sqlite3.connect(DB_NAME)
row = conn.execute("""
    SELECT first_name, last_name, dob, sex, phone, email, address,
           allergies, conditions, medications, notes, emergency_contact
    FROM Patient WHERE patient_id=?
""", (self._selected_patient_id,)).fetchone()
conn.close()
```

```
conn = sqlite3.connect(DB_NAME)
conn.execute("""
    UPDATE Patient SET
        first_name=?, last_name=?, dob=?, sex=?, phone=?, email=?,
        address=?, allergies=?, conditions=?, medications=?, notes=?, emergency_contact=?
    WHERE patient_id=?
""", (*data.values(), self._selected_patient_id))
conn.commit()
conn.close()
```

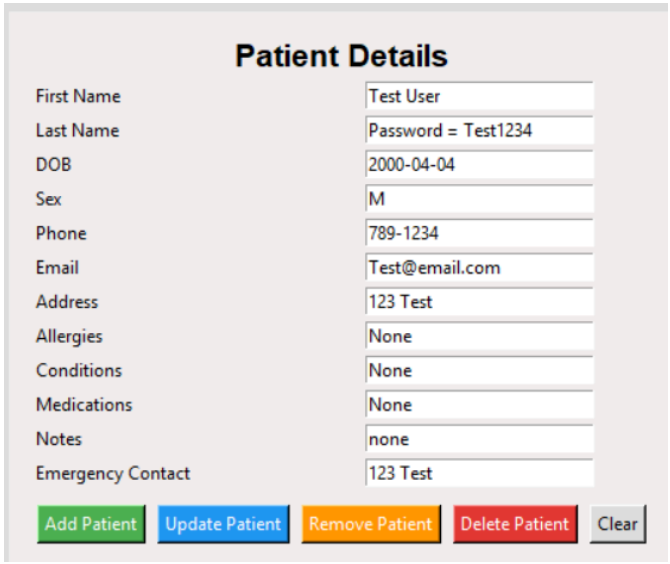
**Use Case 2: Disable Patient (Previously delete patient information)**
**Description: User can disable (soft delete) patient from database, this disables their account and allows it to be rolled back**
**Process:**

1. **Select an existing patient by clicking on them**
2. **Click the yellow "Remove Patient" button on the right hand panel**
3. **A pop up will ask to confirm, click "Yes"**
4. **A pop up will confirm the change, click "OK"**

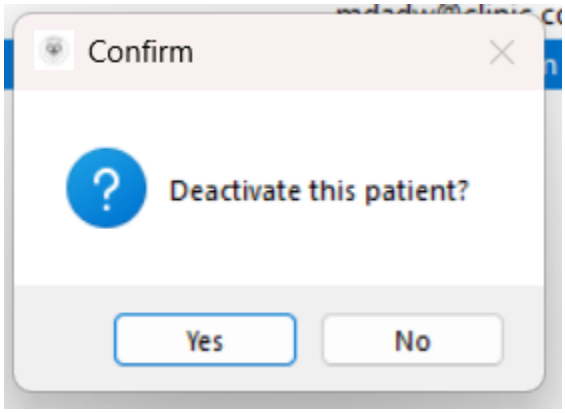
**User case Screenshots:**



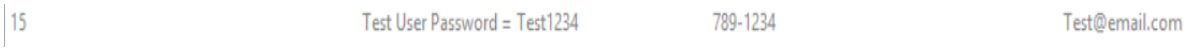

**Select a user to remove.**




**Choose “Remove Patient”**




**Confirm “Yes” and the patient has been removed, or soft deleted**




**Patient is now grayed out**

**Database:**

| | patient_id INTEGER PRIMARY KEY AUTOINCREMENT | first_name TEXT | last_name TEXT | dob TEXT | sex TEXT | phone TEXT | email TEXT |
|---|---|---|---|---|---|---|---|
| 1 | 1 | John | Doe | 1990-05-10 | M | 555-1111 | john.doe@email.com |
| 2 | 2 | Jane | Smith | 1985-03-22 | F | 555-2222 | jane.smith@email.com |
| 3 | 3 | Michael | Brown | 1978-11-02 | M | 555-3333 | mike.b@email.com |
| 4 | 4 | Emily | Davis | 1995-07-18 | F | 555-4444 | emily.d@email.com |
| 5 | 5 | Daniel | Wilson | 1988-01-30 | M | 555-5555 | dan.w@email.com |
| 6 | 6 | Olivia | Taylor | 2001-09-12 | F | 555-6666 | olivia.t@email.com |
| 7 | 7 | James | Anderson | 1969-04-25 | M | 555-7777 | j.anderson@email.com |
| 8 | 8 | Sophia | Thomas | 1999-12-15 | F | 555-8888 | sophia.t@email.com |
| 9 | 9 | William | Jackson | 1975-06-08 | M | 555-9999 | will.j@email.com |
| 10 | 10 | Ava | White | 1993-02-27 | F | 555-0000 | ava.w@email.com |
| 11 | 14 | Manuel | Mora | 2003-01-24 | M | 810-3233 | mdadw@clinic.com |
| 12 | 15 | Test User | Password = Test1234 | 2000-04-04 | M | 789-1234 | Test@email.com |

**Queries:**

```
conn = sqlite3.connect(DB_NAME)
row = conn.execute("""
    SELECT first_name, last_name, dob, sex, phone, email, address,
           allergies, conditions, medications, notes, emergency_contact
    FROM Patient WHERE patient_id=?
""", (self._selected_patient_id,)).fetchone()
conn.close()
```

```
conn = sqlite3.connect(DB_NAME)
cur = conn.cursor()

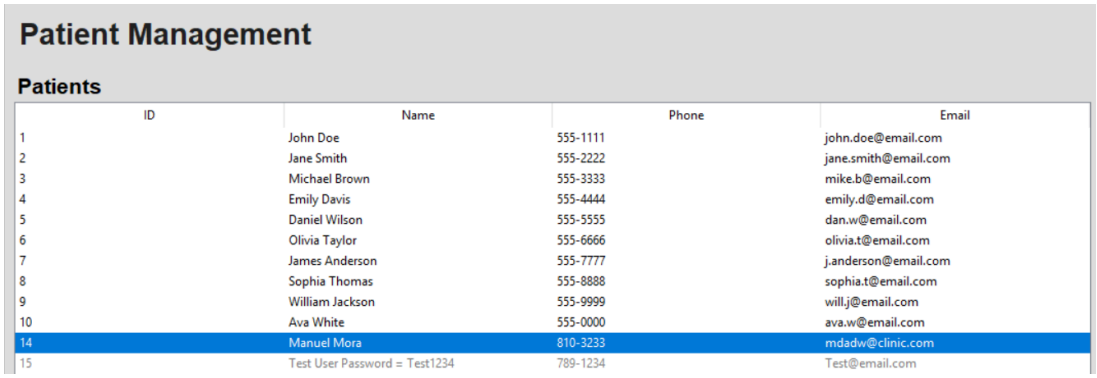
cur.execute("""
    UPDATE Patient
    SET active_flag = 0
    WHERE patient_id = ?
""", (self._selected_patient_id,))

conn.commit()
conn.close()
```

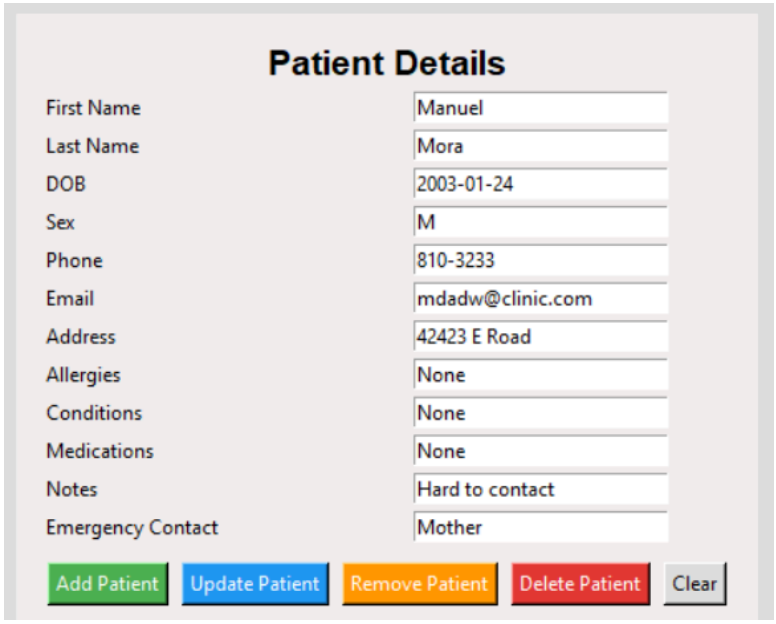
**Use Case 3: Delete User**
**Description: User can fully delete patient from database**
**Process:**

1. **Select an existing patient by clicking on them**
2. **Click the red “Delete Button”**
3. **A pop up will ask to confirm, click “Yes”**
4. **A pop up will confirm the deletion, click “OK”**

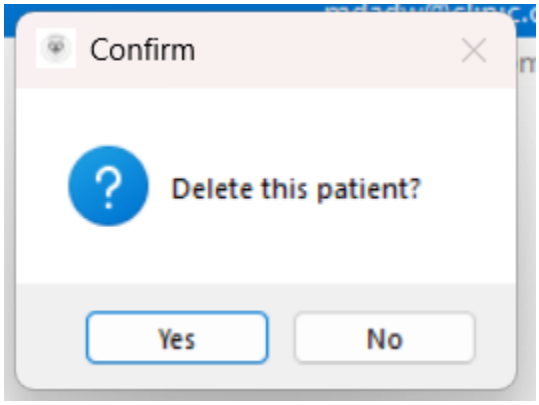
**User case Screenshots:**



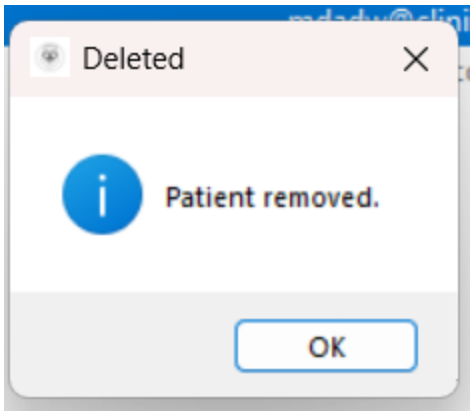

**Choose a patient**




**Select the "Delete Patient" button**




**Confirm by clicking "Yes"**




**Confirm again by clicking “OK”**

**Database:**

| | patient_id INTEGER PRIMARY KEY AUTOINCREMENT | first_name TEXT | last_name TEXT | dob TEXT | sex TEXT | phone TEXT | email TEXT |
|---|---|---|---|---|---|---|---|
| 1 | 1 | John | Doe | 1990-05-10 | M | 555-1111 | john.doe@email.com |
| 2 | 2 | Jane | Smith | 1985-03-22 | F | 555-2222 | jane.smith@email.com |
| 3 | 3 | Michael | Brown | 1978-11-02 | M | 555-3333 | mike.b@email.com |
| 4 | 4 | Emily | Davis | 1995-07-18 | F | 555-4444 | emily.d@email.com |
| 5 | 5 | Daniel | Wilson | 1988-01-30 | M | 555-5555 | dan.w@email.com |
| 6 | 6 | Olivia | Taylor | 2001-09-12 | F | 555-6666 | olivia.t@email.com |
| 7 | 7 | James | Anderson | 1969-04-25 | M | 555-7777 | j.anderson@email.com |
| 8 | 8 | Sophia | Thomas | 1999-12-15 | F | 555-8888 | sophia.t@email.com |
| 9 | 9 | William | Jackson | 1975-06-08 | M | 555-9999 | will.j@email.com |
| 10 | 10 | Ava | White | 1993-02-27 | F | 555-0000 | ava.w@email.com |
| 11 | 14 | Manuel | Mora | 2003-01-24 | M | 810-3233 | mdadw@clinic.com |
| 12 | 15 | Test User | Password = Test1234 | 2000-04-04 | M | 789-1234 | Test@email.com |

**Queries:**

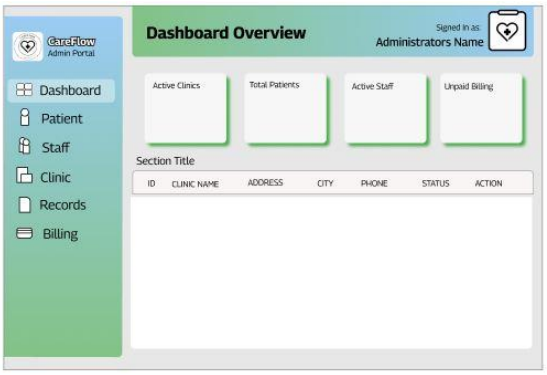
```
conn = sqlite3.connect(DB_NAME)
row = conn.execute("""
    SELECT first_name, last_name, dob, sex, phone, email, address,
           allergies, conditions, medications, notes, emergency_contact
    FROM Patient WHERE patient_id=?
""", (self._selected_patient_id,)).fetchone()
conn.close()
```

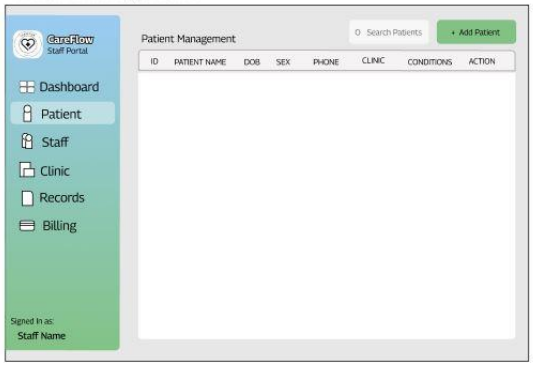
```
conn = sqlite3.connect(DB_NAME)
conn.execute("DELETE FROM Patient WHERE patient_id=?", (self._selected_patient_id,))
conn.commit()
conn.close()
```

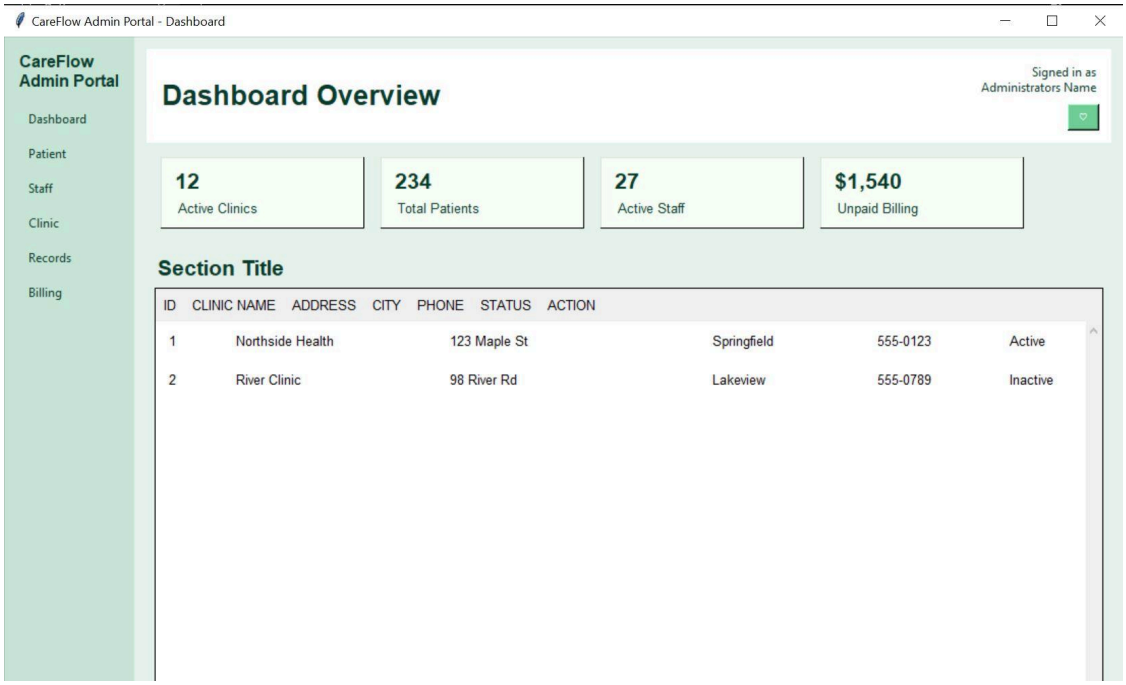
**Video: https://youtu.be/66BdbvRiBxY**

**Notes:**

- **Remove patient greys out text in row, this may be replaced with an active column like in staff management**
- **Major focus on refactoring code, mainly in the menu page**
- **Adapted GUI layout design and began creating a master GUI file for the group to use when building the front end for the last progress report.**

- **GUI updated design using FIGMA**
- 



- **Current progress on dashboard master file**
- 

- **Will continue developing details and then use to complete front end**

**PR5 Complete.**

**PR4: Completed Features for Staff Subsystem**

**PR4: Finished Record System**

Staff Subsystem final use case completed:

**Use Case 1: Update Staff**

**Description:**

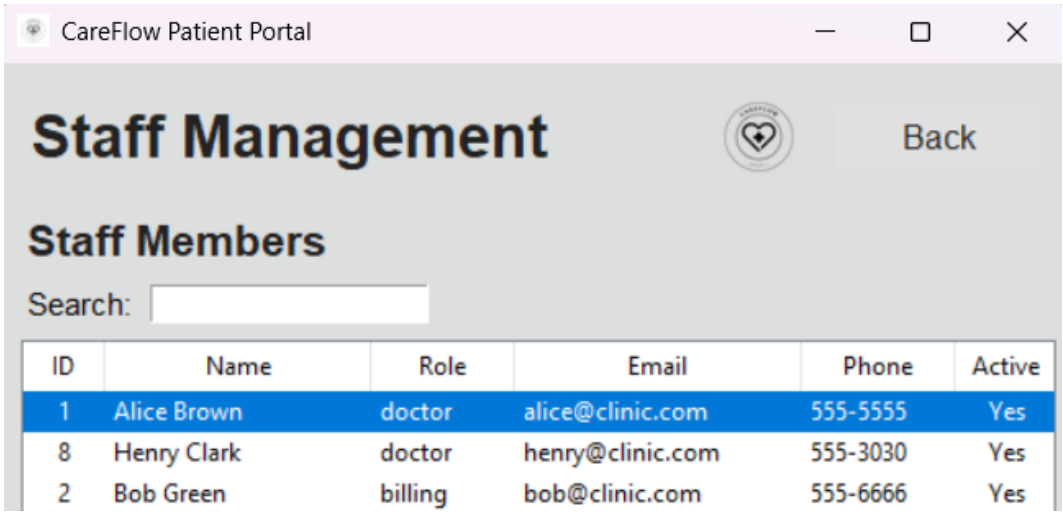
This feature allows staff information to be modified in the system, including personal details, role, active status, and optional password updates. It also updates clinic assignments associated with the staff member. The system first removes all associated clinic assignments from the StaffLocationAssignment table to maintain database integrity. It then deletes the staff member from the Staff table.

**Process:**

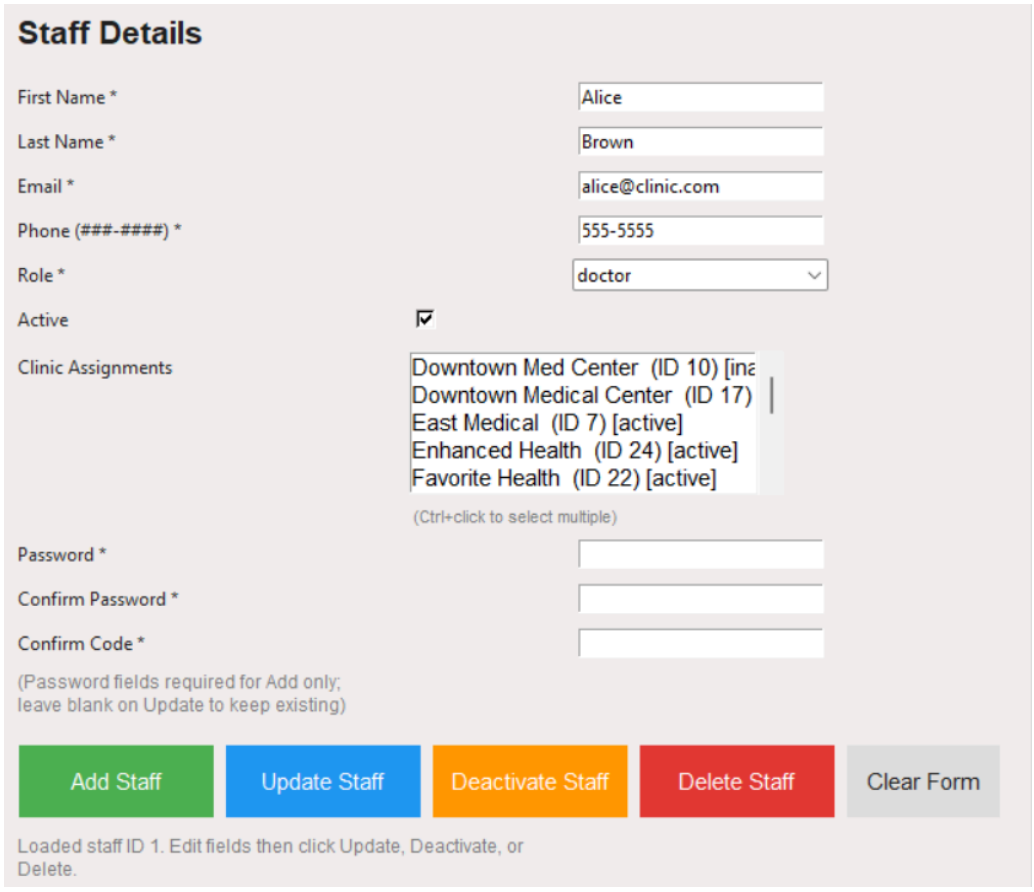
1. User selects a staff member from the list
2. System loads existing staff data into the form

3. User edits desired fields (name, email, phone, role, etc.)
4. System validates input
5. System updates staff information in the database
6. System synchronizes clinic assignments
7. Updated data is displayed in the interface

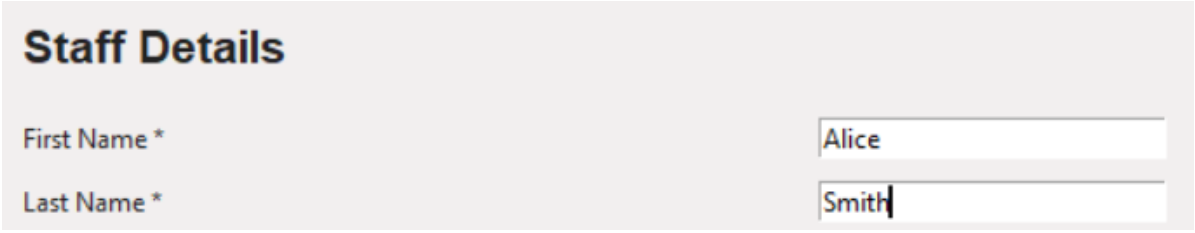
**User case Screenshots:**



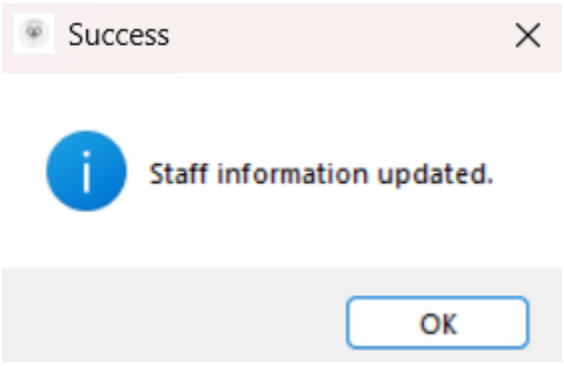

**Select a staff member**



**View current staff details**




**Edit staff name**




**System validates input**

| | staff_id *INTEGER PRIMARY KEY AUTOINCREMENT* | first_name *TEXT* | last_name *TEXT* |
|---|---|---|---|
| 1 | 1 | Alice | Smith |

**The database is updated**

| | | | |
|---|---|---|---|
| 1 | Alice Smith | doctor | alice@clinic.com |

**The interface displays the changes**

**Database:**

SQL 1*

```
SELECT s.staff_id, s.first_name, s.last_name, s.role, s.active_flag,
       sla.location_id, sla.start_date, sla.end_date
FROM Staff s
```

| | staff_id | first_name | last_name | role | active_flag | location_id | start_date | end_date |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Alice | Brown | doctor | 1 | 1 | 2024-01-01 | NULL |
| 2 | 2 | Bob | Green | billing | 1 | 2 | 2024-02-01 | NULL |
| 3 | 3 | Carol | White | records | 1 | 3 | 2024-03-01 | NULL |
| 4 | 4 | David | Lee | patients | 1 | 4 | 2024-04-01 | NULL |
| 5 | 5 | Emma | Taylor | billing | 1 | 5 | 2024-05-01 | NULL |
| 6 | 5 | Emma | Taylor | billing | 1 | 7 | 2026-03-13 | NULL |
| 7 | 5 | Emma | Taylor | billing | 1 | 17 | 2026-03-13 | NULL |
| 8 | 6 | Frank | Harris | records | 1 | 6 | 2024-06-01 | NULL |
| 9 | 7 | Grace | Martin | patients | 1 | 7 | 2024-07-01 | NULL |
| 10 | 8 | Henry | Clark | doctor | 1 | 8 | 2024-08-01 | NULL |
| 11 | 9 | Ivy | Walker | billing | 1 | 9 | 2024-09-01 | NULL |
| 12 | 10 | Jack | Hall | records | 1 | 10 | 2024-10-01 | NULL |
| 13 | 11 | Test | Test | doctor | 1 | NULL | NULL | NULL |
| 14 | 12 | test2 | tester | records | 1 | NULL | NULL | NULL |
| 15 | 13 | Fred | John | doctor | 1 | 23 | 2026-03-13 | NULL |

**Queries:**

```
        return
    pw_hash = hash_password(pw)
    sql = "UPDATE Staff SET first_name=?, last_name=?, email=?, phone=?, role=?, active_flag=?, password_hash=? WHERE staff_id=?"
    params = (data["first_name"], data["last_name"], data["email"], data["phone"], data["role"], data["active"], pw_hash, self._selected_s
else:
    sql = "UPDATE Staff SET first_name=?, last_name=?, email=?, phone=?, role=?, active_flag=? WHERE staff_id=?"
    params = (data["first_name"], data["last_name"], data["email"], data["phone"], data["role"], data["active"], self._selected_staff_id)
try:
    conn = sqlite3.connect(DB_NAME)
    conn.execute(sql, params)
    # Sync clinic assignments
    current_locs = {r[0] for r in conn.execute(
        "SELECT location_id FROM StaffLocationAssignment WHERE staff_id=? AND end_date IS NULL",
```

```
        "INSERT INTO StaffLocationAssignment (staff_id, location_id, assignment_role, start_date) VALUES (?,?,?,date('now'))",
        (self._selected_staff_id, loc_id, data["role"])
    )
for loc_id in current_locs - new_loc_ids:
    conn.execute(
        "UPDATE StaffLocationAssignment SET end_date=date('now') WHERE staff_id=? AND location_id=? AND end_date IS NULL",
        (self._selected_staff_id, loc_id)
    )
```

**PR4 Staff Feature:**

**Use Case 2: Delete Staff**

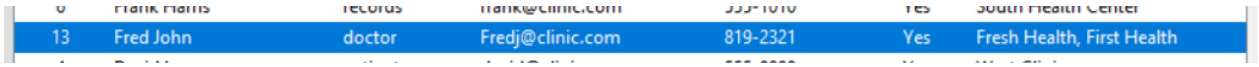
**Description:**

This feature permanently removes a staff member from the system/database along with their clinic assignments.

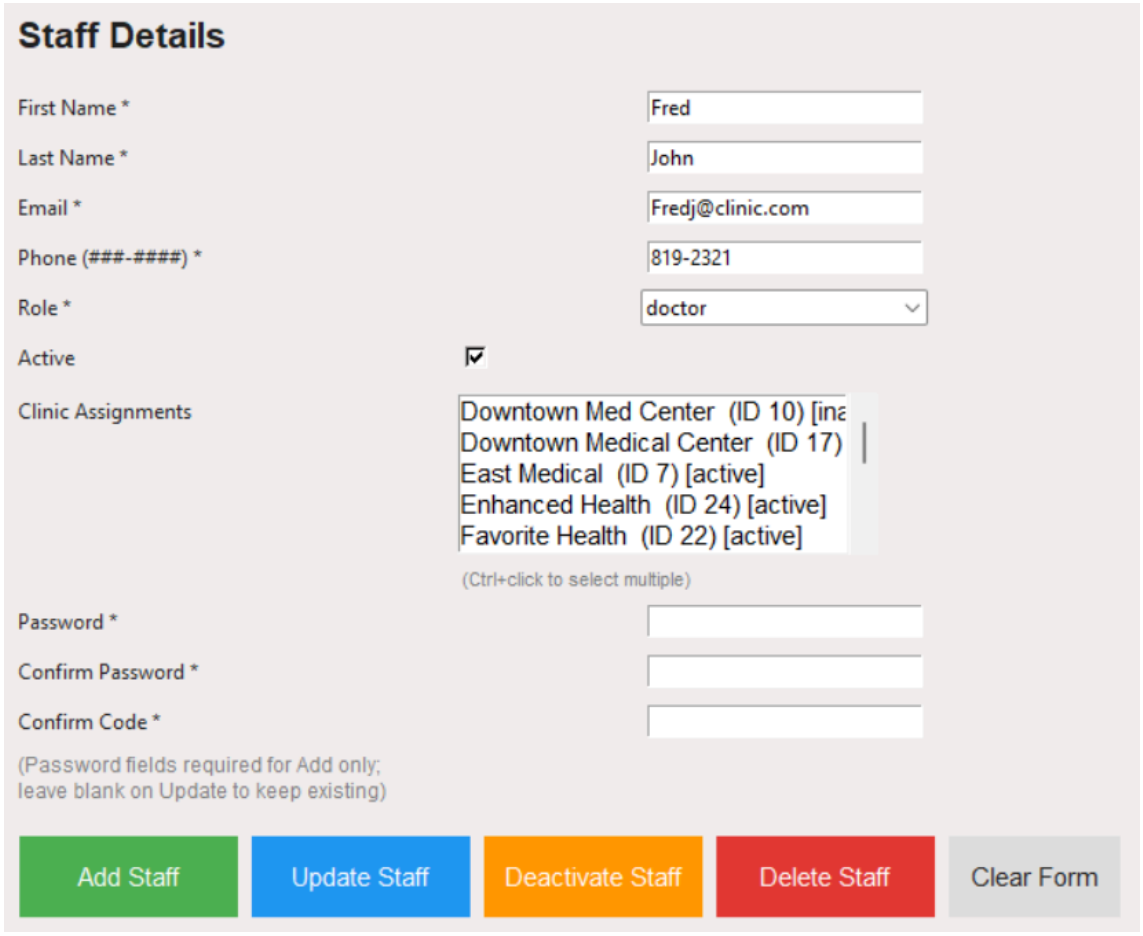
**Process:**

1. User selects a staff member
2. User clicks "Delete Staff"
3. System asks for confirmation
4. System deletes staff assignments
5. System deletes staff record
6. System refreshes staff list

**User case Screenshots:**



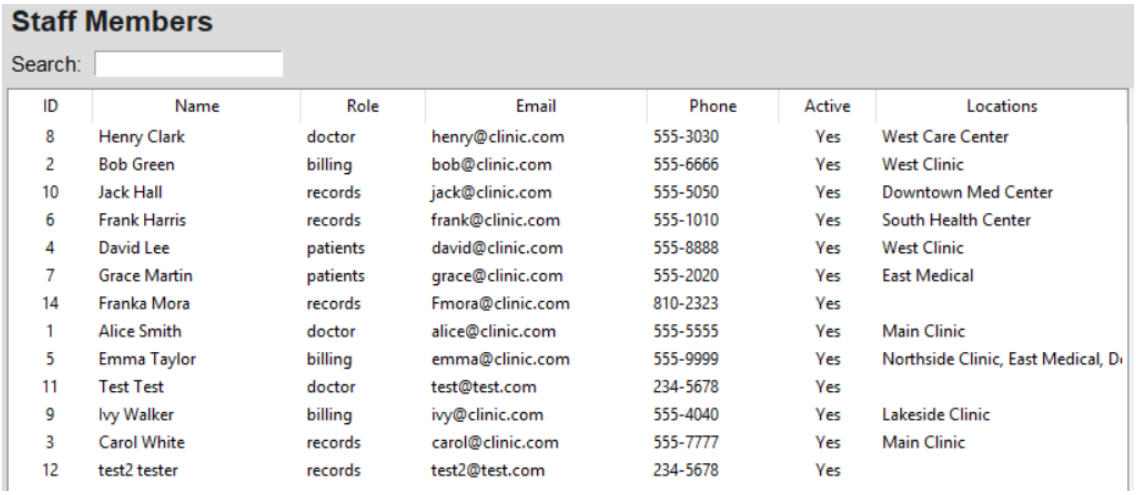

**Select staff member**




**User chooses to delete staff**




**System asks for conformation**



Staff Members

Search:

| ID | Name | Role | Email | Phone | Active | Locations |
|---|---|---|---|---|---|---|
| 8 | Henry Clark | doctor | henry@clinic.com | 555-3030 | Yes | West Care Center |
| 2 | Bob Green | billing | bob@clinic.com | 555-6666 | Yes | West Clinic |
| 10 | Jack Hall | records | jack@clinic.com | 555-5050 | Yes | Downtown Med Center |
| 6 | Frank Harris | records | frank@clinic.com | 555-1010 | Yes | South Health Center |
| 4 | David Lee | patients | david@clinic.com | 555-8888 | Yes | West Clinic |
| 7 | Grace Martin | patients | grace@clinic.com | 555-2020 | Yes | East Medical |
| 14 | Franka Mora | records | Fmora@clinic.com | 810-2323 | Yes | |
| 1 | Alice Smith | doctor | alice@clinic.com | 555-5555 | Yes | Main Clinic |
| 5 | Emma Taylor | billing | emma@clinic.com | 555-9999 | Yes | Northside Clinic, East Medical, D |
| 11 | Test Test | doctor | test@test.com | 234-5678 | Yes | |
| 9 | Ivy Walker | billing | ivy@clinic.com | 555-4040 | Yes | Lakeside Clinic |
| 3 | Carol White | records | carol@clinic.com | 555-7777 | Yes | Main Clinic |
| 12 | test2 tester | records | test2@test.com | 234-5678 | Yes | |

**Staff record has been removed and interface updated**

**Database:**

**Before:**

```
1  SELECT *
2  FROM Staff;
3
```

| | staff_id | first_name | last_name | email | phone | role | active_flag | |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Alice | Brown | alice@clinic.com | 555-5555 | doctor | 1 | NULL |
| 2 | 2 | Bob | Green | bob@clinic.com | 555-6666 | billing | 1 | NULL |
| 3 | 3 | Carol | White | carol@clinic.com | 555-7777 | records | 1 | NULL |
| 4 | 4 | David | Lee | david@clinic.com | 555-8888 | patients | 1 | NULL |
| 5 | 5 | Emma | Taylor | emma@clinic.com | 555-9999 | billing | 1 | NULL |
| 6 | 6 | Frank | Harris | frank@clinic.com | 555-1010 | records | 1 | NULL |
| 7 | 7 | Grace | Martin | grace@clinic.com | 555-2020 | patients | 1 | NULL |
| 8 | 8 | Henry | Clark | henry@clinic.com | 555-3030 | doctor | 1 | NULL |
| 9 | 9 | Ivy | Walker | ivy@clinic.com | 555-4040 | billing | 1 | NULL |
| 10 | 10 | Jack | Hall | jack@clinic.com | 555-5050 | records | 1 | NULL |
| 11 | 11 | Test | Test | test@test.com | 234-5678 | doctor | 1 | pbkdf2 |
| 12 | 12 | test2 | tester | test2@test.com | 234-5678 | records | 1 | pbkdf2 |
| 13 | 13 | Fred | John | Fredj@clinic.com | 819-2321 | doctor | 1 | pbkdf2 |
| 14 | 14 | Franka | Mora | Fmora@clinic.com | 810-2323 | records | 0 | pbkdf2 |

**After:**

| | staff_id | first_name | last_name | email | phone | role | active_flag | |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Alice | Brown | alice@clinic.com | 555-5555 | doctor | 1 | NULL |
| 2 | 2 | Bob | Green | bob@clinic.com | 555-6666 | billing | 1 | NULL |
| 3 | 3 | Carol | White | carol@clinic.com | 555-7777 | records | 1 | NULL |
| 4 | 4 | David | Lee | david@clinic.com | 555-8888 | patients | 1 | NULL |
| 5 | 5 | Emma | Taylor | emma@clinic.com | 555-9999 | billing | 1 | NULL |
| 6 | 6 | Frank | Harris | frank@clinic.com | 555-1010 | records | 1 | NULL |
| 7 | 7 | Grace | Martin | grace@clinic.com | 555-2020 | patients | 1 | NULL |
| 8 | 8 | Henry | Clark | henry@clinic.com | 555-3030 | doctor | 1 | NULL |
| 9 | 9 | Ivy | Walker | ivy@clinic.com | 555-4040 | billing | 1 | NULL |
| 10 | 10 | Jack | Hall | jack@clinic.com | 555-5050 | records | 1 | NULL |
| 11 | 11 | Test | Test | test@test.com | 234-5678 | doctor | 1 | pbkdf2_sha |
| 12 | 12 | test2 | tester | test2@test.com | 234-5678 | records | 1 | pbkdf2_sha |
| 13 | 13 | Fred | John | Fredj@clinic.com | 819-2321 | doctor | 1 | pbkdf2_sha |

**Queries:**

```
try:
    conn = sqlite3.connect(DB_NAME)
    conn.execute("DELETE FROM StaffLocationAssignment WHERE staff_id=?", (self._selected_staff_id,))
    conn.execute("DELETE FROM Staff WHERE staff_id=?", (self._selected_staff_id,))
    conn.commit()
    conn.close()
```

**PR4 Staff Feature:**

**Use Case 3: Disable Staff**

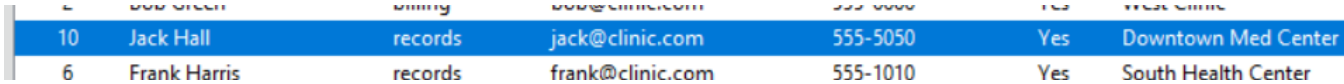
**Description:**

This feature allows a staff member to be disabled without deleting their data from the system. This is useful for maintaining historical records while preventing access. The query performs a soft delete by setting the active_flag to 0. The staff member remains in the database, but is considered inactive.

**Process:**

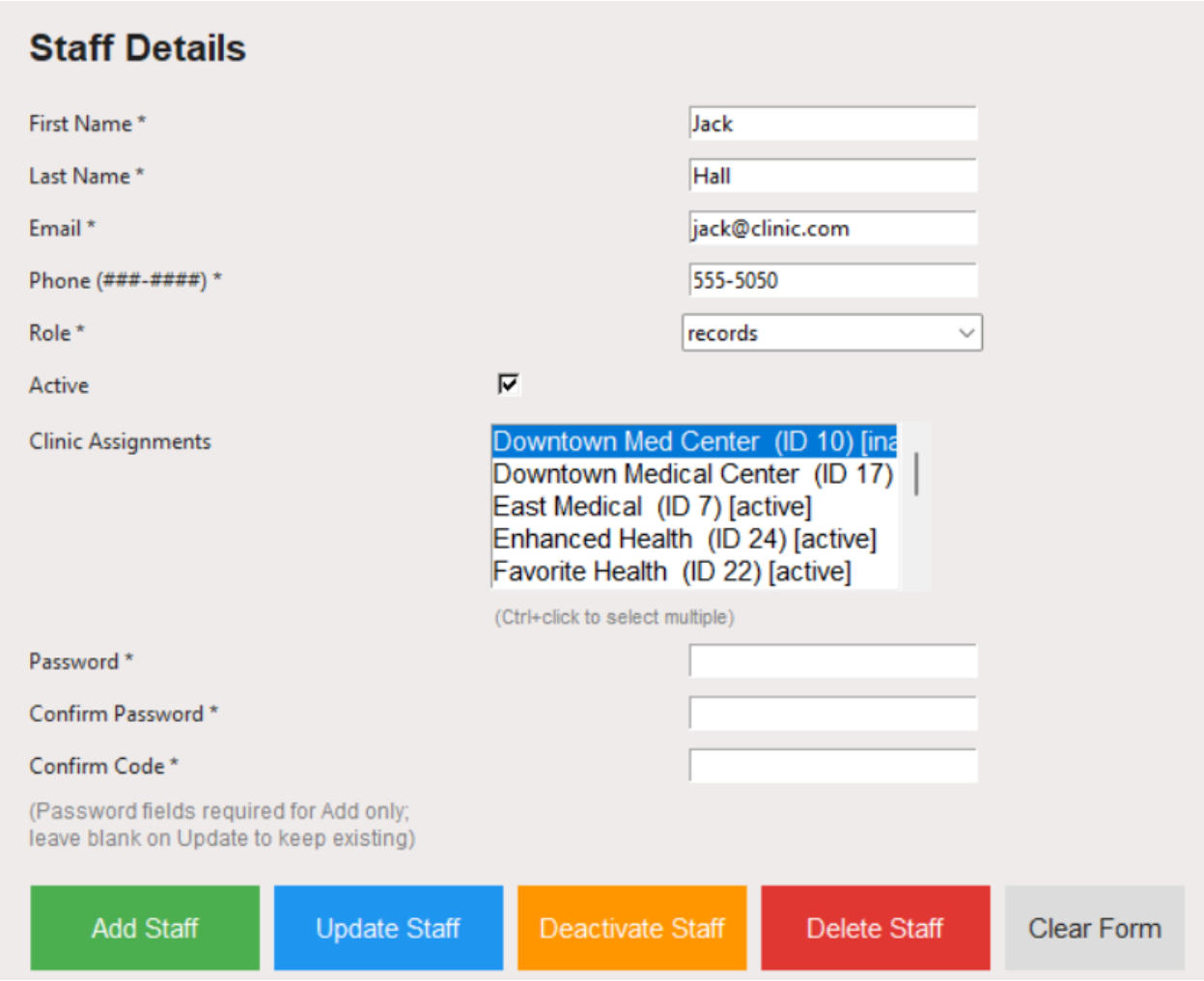
1. User selects a staff member
2. User clicks "Deactivate Staff"
3. System asks for confirmation
4. System updates staff status
5. Staff member is marked as inactive
6. System refreshes the interface

**Use case Screenshots:**

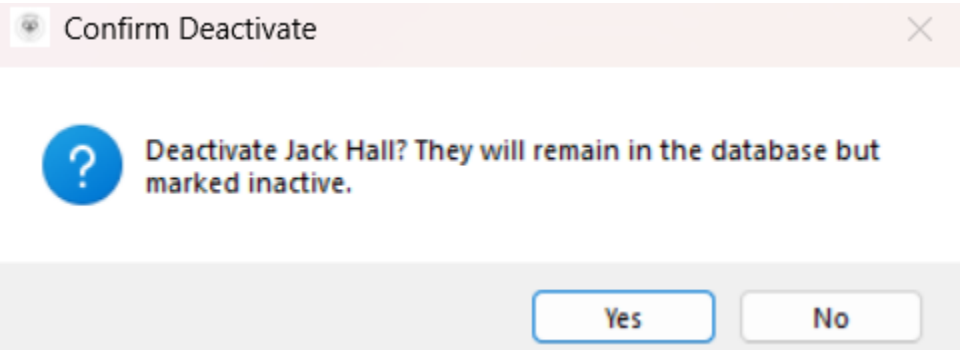


| | | | | | | |
|---|---|---|---|---|---|---|
| 10 | Jack Hall | records | jack@clinic.com | 555-5050 | Yes | Downtown Med Center |
| 6 | Frank Harris | records | frank@clinic.com | 555-1010 | Yes | South Health Center |

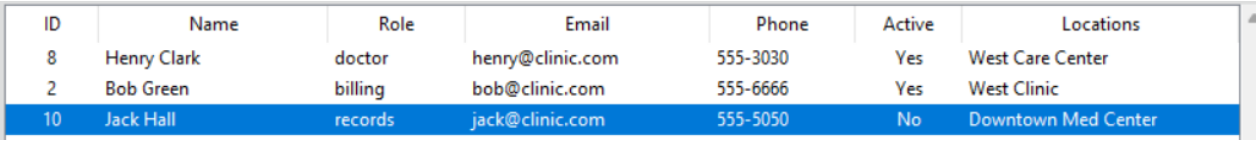
**User selects a staff member**




**User chooses to deactivate staff**




**System prompts confirmation**



| ID | Name | Role | Email | Phone | Active | Locations |
|---|---|---|---|---|---|---|
| 8 | Henry Clark | doctor | henry@clinic.com | 555-3030 | Yes | West Care Center |
| 2 | Bob Green | billing | bob@clinic.com | 555-6666 | Yes | West Clinic |
| 10 | Jack Hall | records | jack@clinic.com | 555-5050 | No | Downtown Med Center |

**System updates and marks staff as inactive**

**Database:**

SQL 1*

```
SELECT staff_id, first_name, last_name, role, active_flag
FROM Staff;
```

| | staff_id | first_name | last_name | role | active_flag |
|---|---|---|---|---|---|
| 1 | 1 | Alice | Brown | doctor | 1 |
| 2 | 2 | Bob | Green | billing | 1 |
| 3 | 3 | Carol | White | records | 1 |
| 4 | 4 | David | Lee | patients | 1 |
| 5 | 5 | Emma | Taylor | billing | 1 |
| 6 | 6 | Frank | Harris | records | 1 |
| 7 | 7 | Grace | Martin | patients | 1 |
| 8 | 8 | Henry | Clark | doctor | 1 |
| 9 | 9 | Ivy | Walker | billing | 1 |
| 10 | 10 | Jack | Hall | records | 1 |
| 11 | 11 | Test | Test | doctor | 1 |
| 12 | 12 | test2 | tester | records | 1 |
| 13 | 13 | Fred | John | doctor | 1 |
| 14 | 14 | Franka | Mora | records | 0 |

**Queries:**

```
def _remove_staff(self):
    """Soft delete — sets active_flag = 0, keeps the record and assignments in the DB."""
    if not self._selected_staff_id:
        messagebox.showwarning("No Selection", "Select a staff member from the list first.")
        return
    name = self._get_selected_name()
    if not messagebox.askyesno("Confirm Deactivate", f"Deactivate {name}? They will remain in the database but marked inactive."):
        return
    try:
        conn = sqlite3.connect(DB_NAME)
        conn.execute("UPDATE Staff SET active_flag=0 WHERE staff_id=?", (self._selected_staff_id,))
        conn.commit()
        conn.close()
```

**PR4 Video:** https://youtu.be/Yi-Ur7ZDrrs

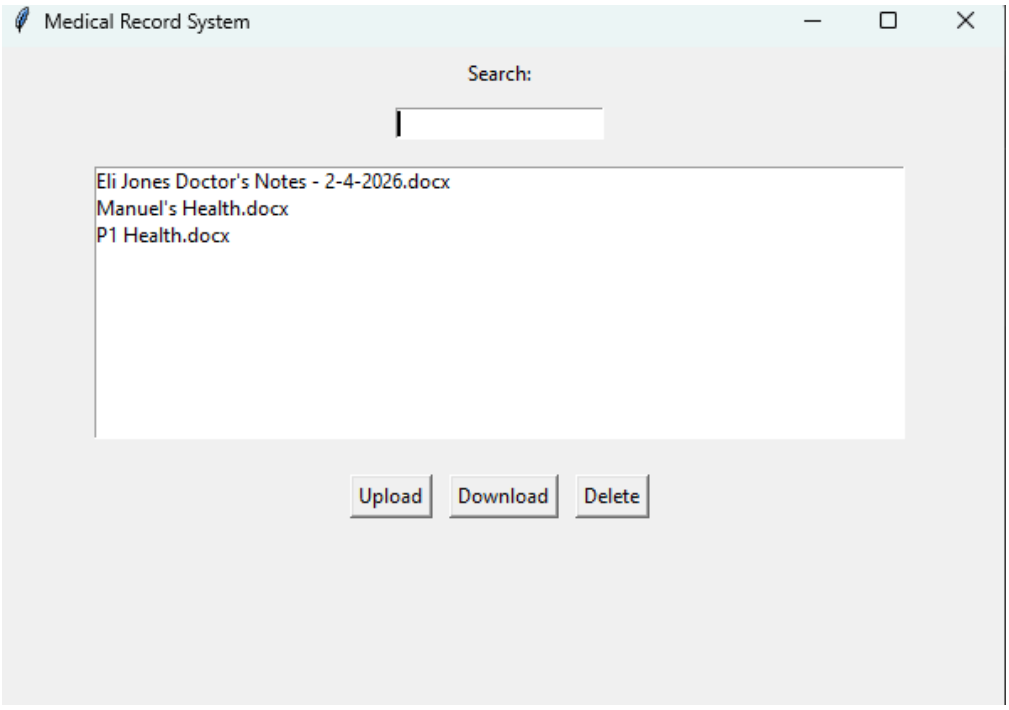
**PR3: Finished Record System and Implementing Location Management Subsystem**
**PR3: Finished Record System**
Patient Record System final use case is completed:

- Upload patient files
- Download files
- Delete files
- Search Files

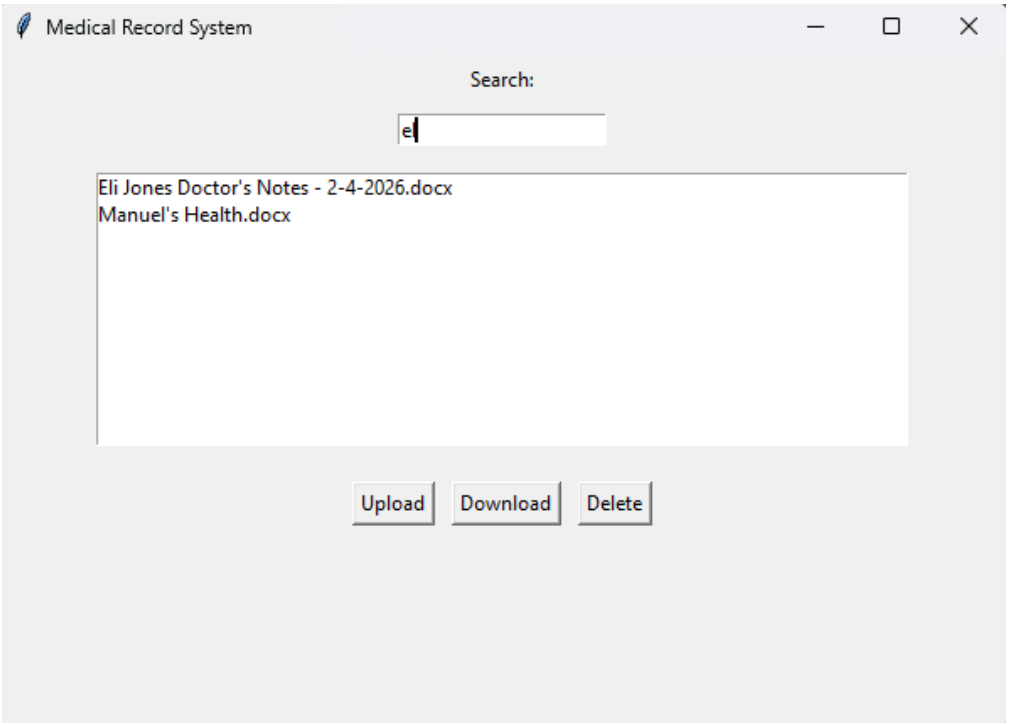
**Use Case 1: Search Files**
**Description:**
This use case allows users to use a search bar to search for a specific record file within the records table. When the user types the name, the listbox will refresh, filtering out filenames not relevant to what they are typing. The listbox will update live as they type.

**Process:**
1. The user click on the search box above the listboc
2. The user types the filename or terms relevant to the filename
3. The listbox updates live to match the user's input

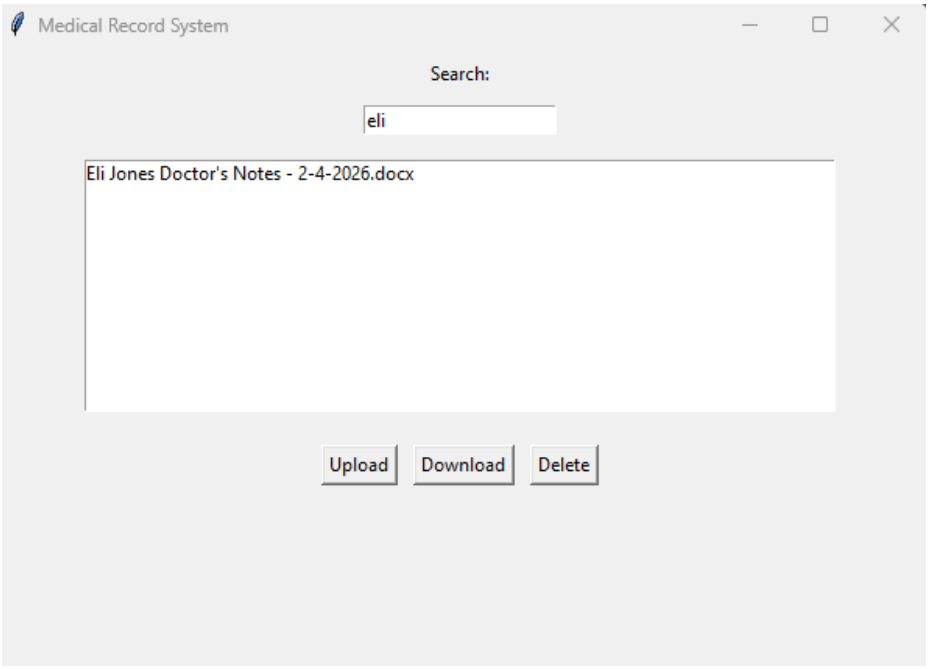
**User Clicks on Searchbox:**




**User Types Final Name and Listbox Filters**




**Listbox Updates to Match User's Input**




**Database:**

| id | filename | filepath | upload_date |
|---|---|---|---|
| 1 | Patient Info.pdf | record_files\Patient Info.pdf | 2026-02-16 10:34:41 |
| 2 | Manuel's Health .pdf | record_files\Manuel's Health .pdf | 2026-02-16 10:34:47 |
| 3 | Allergies Health .pdf | record_files\Allergies Health .pdf | 2026-02-16 10:34:56 |
| 4 | P1 Health.pdf | record_files\P1 Health.pdf | 2026-02-16 10:43:52 |
| 5 | P2 Health.pdf | record_files\P2 Health.pdf | 2026-02-16 10:44:15 |
| 6 | P3 Health.pdf | record_files\P3 Health.pdf | 2026-02-16 10:44:33 |
| 7 | P4 Health.pdf | record_files\P4 Health.pdf | 2026-02-16 10:44:46 |
| 8 | P5 Health.pdf | record_files\P5 Health.pdf | 2026-02-16 10:44:58 |
| 9 | P6 Health.pdf | record_files\P6 Health.pdf | 2026-02-16 10:45:23 |
| 10 | P67 Health.pdf | record_files\P67 Health.pdf | 2026-02-16 10:45:42 |

**Search Files**

```
def update_file_list(self, event=None):
    search_term = self.search_var.get().lower()
    self.file_listbox.delete(0, tk.END)

    cursor.execute("SELECT filename FROM records")
    files = cursor.fetchall()

    for file in files:
        filename = file[0]
        if search_term in filename.lower():
            self.file_listbox.insert(tk.END, filename)
```

Location Management System first three use cases implemented:

- Add new clinic location
- Update clinic location
- Delete/ Remove clinic location
- Attach Doctor to location
- Remove Doctor from location

**Use Case 1: Add new clinic location**

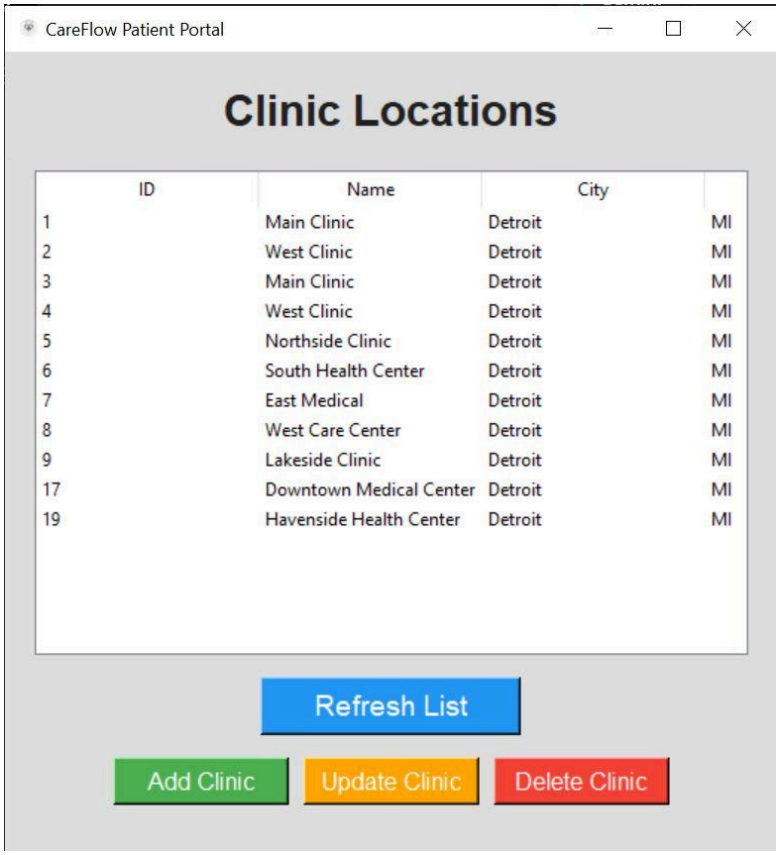
**Description:**

This use case allows authorized users to create and store a new clinic location record in the system. This ensures that new clinic locations can be tracked, managed, and accessed within the application.

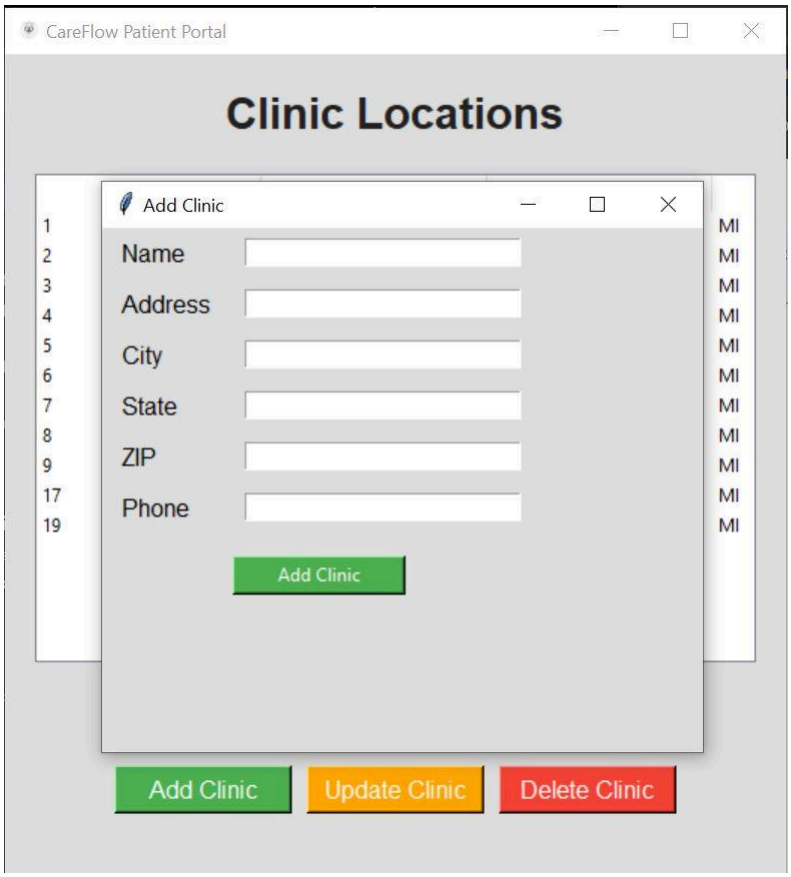
**Process:**

1. User selects the Add clinic button
2. The system prompts user to enter required clinic details (clinic name, address, phone number, services, etc.)
3. User inputs the required information
4. System validates the input
5. System saves the new clinic record to the database
6. System displays a confirmation message indicating the clinic was added successfully

**Step 1 screenshot - Add clinic button click**



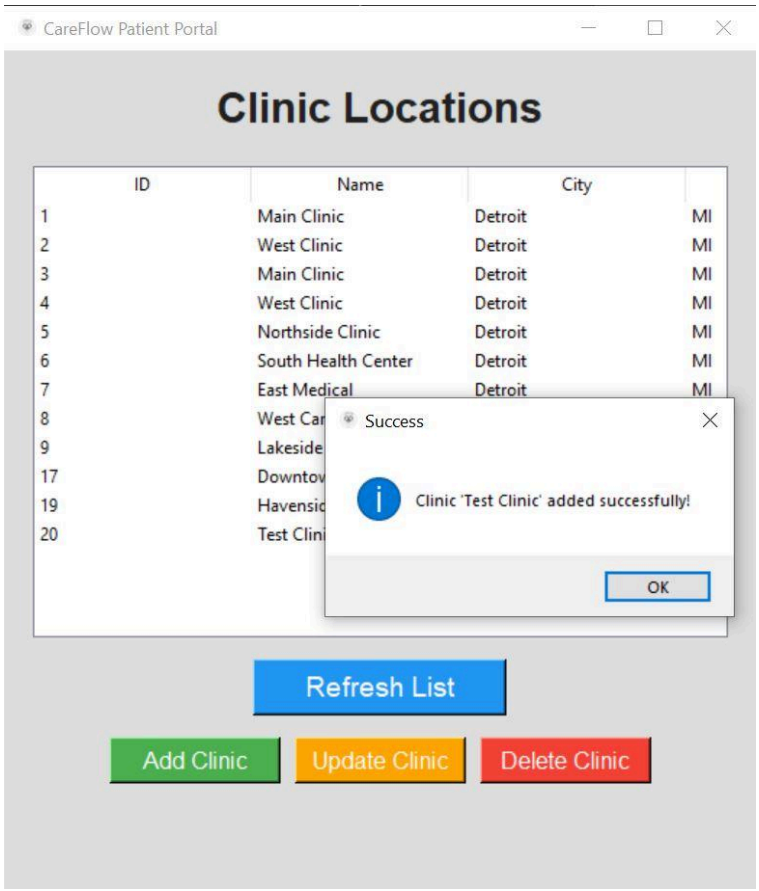

**Step 2 screenshot - User prompt**




**Step 3 screenshot - User input**




**Step 6 screenshot - Confirmation message**




**Database screenshot**

| | location_id *INTEGER PRIMARY KEY AUTOINCREMENT* | name *TEXT NOT NULL* | address *TEXT NOT NULL* | city *TEXT NOT NULL* | state *TEXT NOT NULL* | zip *TEXT NOT NULL* | phone *TEXT* | status *TEXT DEFAULT* |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 2 | 2 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 3 | 3 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 4 | 4 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 5 | 5 | Northside Clinic | 111 North Rd | Detroit | MI | 48203 | 555-3001 | active |
| 6 | 6 | South Health Center | 222 South Blvd | Detroit | MI | 48204 | 555-3002 | active |
| 7 | 7 | East Medical | 333 East Ave | Detroit | MI | 48205 | 555-3003 | active |
| 8 | 8 | West Care Center | 444 West Dr | Detroit | MI | 48206 | 555-3004 | active |
| 9 | 9 | Lakeside Clinic | 555 Lake Shore | Detroit | MI | 48207 | 555-3005 | active |
| 10 | 10 | Downtown Med Center | Detroit | MI | MI | 48226 | 555-6789 | inactive |
| 11 | 11 | Test Clinic | 123 Test St | Detroit | MI | 48201 | 555-1234 | inactive |
| 12 | 12 | Havenside Clinic | 1234 North St | Detroit | MI | 48272 | | inactive |
| 13 | 13 | Havenside | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 14 | 14 | Havenside Clinic | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 15 | 15 | Havenside Clinic | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 16 | 16 | Haven Medical Center | 1234 Altlas Ln | Detroit | MI | 47008 | | inactive |
| 17 | 17 | Downtown Medical Ce… | 1234 Altas Ln | Detroit | MI | 47006 | | active |
| 18 | 18 | Havenside Clinic | 1236 Cloud Ln | Detroit | MI | 47008 | | inactive |

**Function:**

Dr. Hadeel Jawad, Winter 2026 – CSI 4999

```python
def add_clinic_location(name, address, city, state, zip_code, phone):
    try:
        conn = sqlite3.connect(DB_NAME)
        cur = conn.cursor()

        cur.execute("""
            INSERT INTO ClinicLocation (name, address, city, state, zip, phone, status)
            VALUES (?, ?, ?, ?, ?, ?, 'active')
        """, (name, address, city, state, zip_code, phone))

        conn.commit()
        conn.close()
        return True

    except sqlite3.Error as e:
        messagebox.showerror("Database Error", f"Failed to add clinic:\n\n{e}")
        return False
```

**Use Case 2: Update Clinic Location**

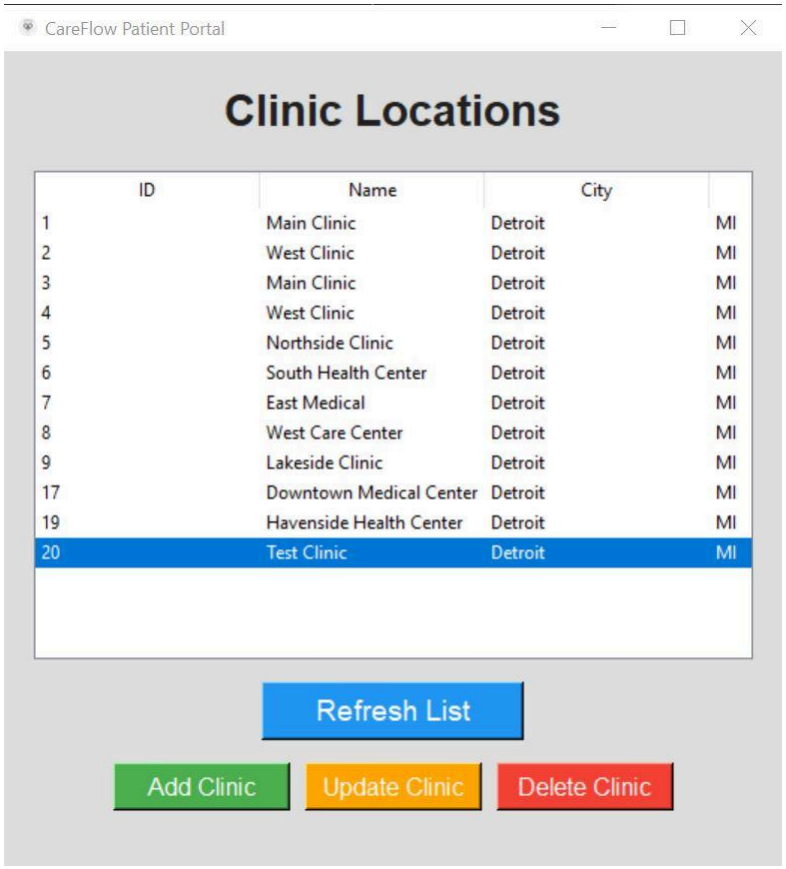
**Description:**

The Update clinic use case allows an authorized user to modify existing clinic information. This ensures that clinic records remain accurate and up to date.

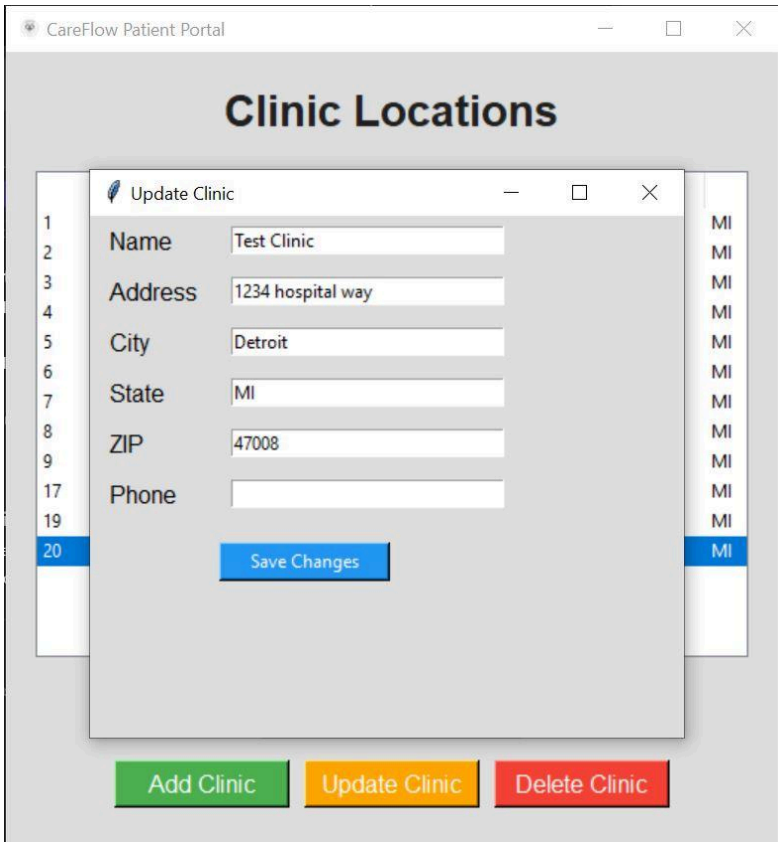
**Process**:

1. System displays a list of existing clinics
2. User selects the clinic to update by clicking listing
3. System displays the current clinic details
4. User enters updated information for one or more fields
5. System validates new data
6. System saves the updated information to the database
7. System displays a confirmation message indicating the update was successful
8. System updates displayed list

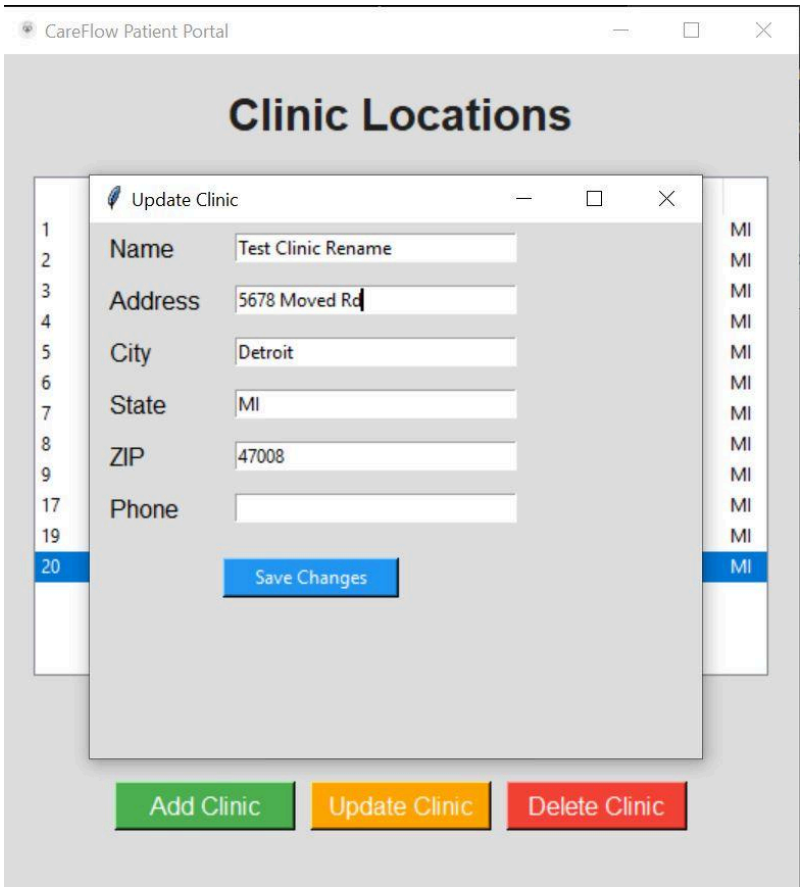
**Step 2 screenshot - Select Clinic to update, then click update button**



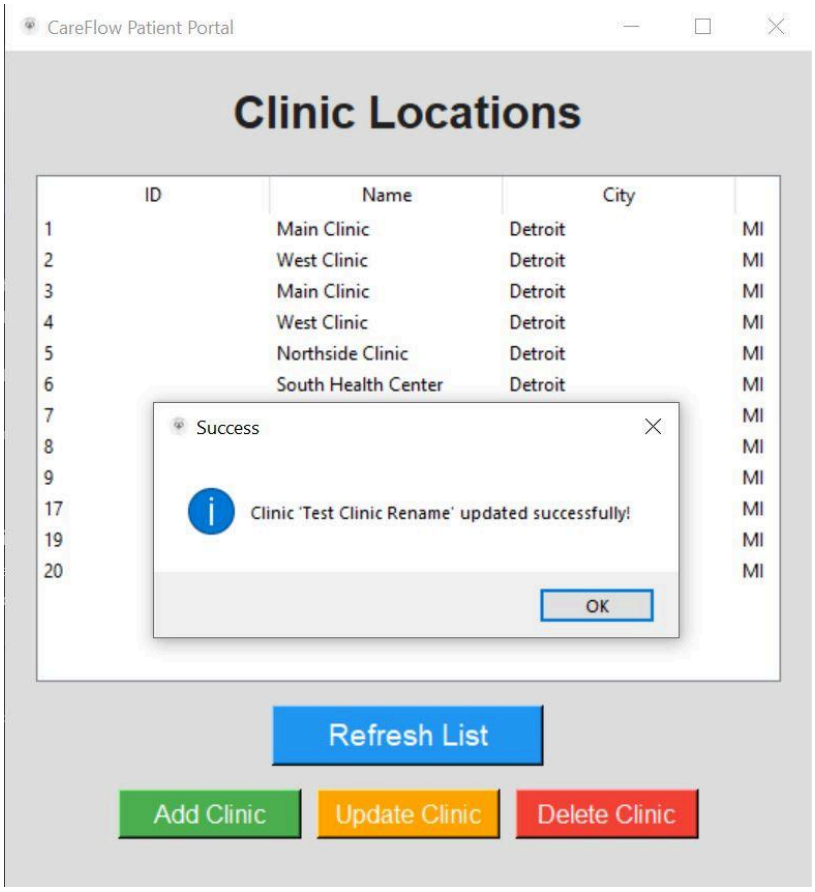

**Step 3 screenshot - Current details displayed**




**Step 4 screenshot - User updates information**




**Step 7 screenshot - Confirmation message**



## Step 8 screenshot - Updated listings

| | | | |
|---|---|---|---|
| 8 | West Care Center | Detroit | MI |
| 9 | Lakeside Clinic | Detroit | MI |
| 17 | Downtown Medical Center | Detroit | MI |
| 19 | Havenside Health Center | Detroit | MI |
| 20 | Test Clinic Rename | Detroit | MI |

## Database screenshot

| | location_id *INTEGER PRIMARY KEY AUTOINCREMENT* | name *TEXT NOT NULL* | address *TEXT NOT NULL* | city *TEXT NOT NULL* | state *TEXT NOT NULL* | zip *TEXT NOT NULL* | phone *TEXT* | status *TEXT DEFAULT* |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 2 | 2 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 3 | 3 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 4 | 4 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 5 | 5 | Northside Clinic | 111 North Rd | Detroit | MI | 48203 | 555-3001 | active |
| 6 | 6 | South Health Center | 222 South Blvd | Detroit | MI | 48204 | 555-3002 | active |
| 7 | 7 | East Medical | 333 East Ave | Detroit | MI | 48205 | 555-3003 | active |
| 8 | 8 | West Care Center | 444 West Dr | Detroit | MI | 48206 | 555-3004 | active |
| 9 | 9 | Lakeside Clinic | 555 Lake Shore | Detroit | MI | 48207 | 555-3005 | active |
| 10 | 10 | Downtown Med Center | Detroit | MI | MI | 48226 | 555-6789 | inactive |
| 11 | 11 | Test Clinic | 123 Test St | Detroit | MI | 48201 | 555-1234 | inactive |
| 12 | 12 | Havenside Clinic | 1234 North St | Detroit | MI | 48272 | | inactive |
| 13 | 13 | Havenside | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 14 | 14 | Havenside Clinic | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 15 | 15 | Havenside Clinic | 1234 Cloud Ln | Detroit | MI | 47007 | | inactive |
| 16 | 16 | Haven Medical Center | 1234 Altlas Ln | Detroit | MI | 47008 | | inactive |
| 17 | 17 | Downtown Medical Ce… | 1234 Altas Ln | Detroit | MI | 47006 | | active |
| 18 | 18 | Havenside Clinic | 1236 Cloud Ln | Detroit | MI | 47008 | | inactive |

**Function:**

```python
def update_clinic_location(location_id, name=None, address=None, city=None, state=None, zip_code=None, phone=None):
    """
    Update clinic fields. Only updates the fields that are not None.
    Returns True if successful, False if error.
    """
    try:
        conn = sqlite3.connect(DB_NAME)
        cur = conn.cursor()

        # Build the SET clause dynamically
        fields = []
        values = []

        if name is not None:
            fields.append("name = ?")
            values.append(name)
        if address is not None:
            fields.append("address = ?")
            values.append(address)
        if city is not None:
            fields.append("city = ?")
            values.append(city)
        if state is not None:
            fields.append("state = ?")
            values.append(state)
        if zip_code is not None:
            fields.append("zip = ?")
            values.append(zip_code)
        if phone is not None:
            fields.append("phone = ?")
            values.append(phone)

        if not fields:
            # Nothing to update
            conn.close()
            return False

        values.append(location_id)
        sql = f"UPDATE ClinicLocation SET {', '.join(fields)} WHERE location_id = ?"
        cur.execute(sql, values)
        conn.commit()
        conn.close()
        return True

    except sqlite3.Error as e:
        messagebox.showerror("Database Error", f"Failed to update clinic:\n\n{e}")
        return False
```

**Video Link: https://youtu.be/q7hkXLeBgpw**
**END PR3**

**PR2: Record Management Subsystem**

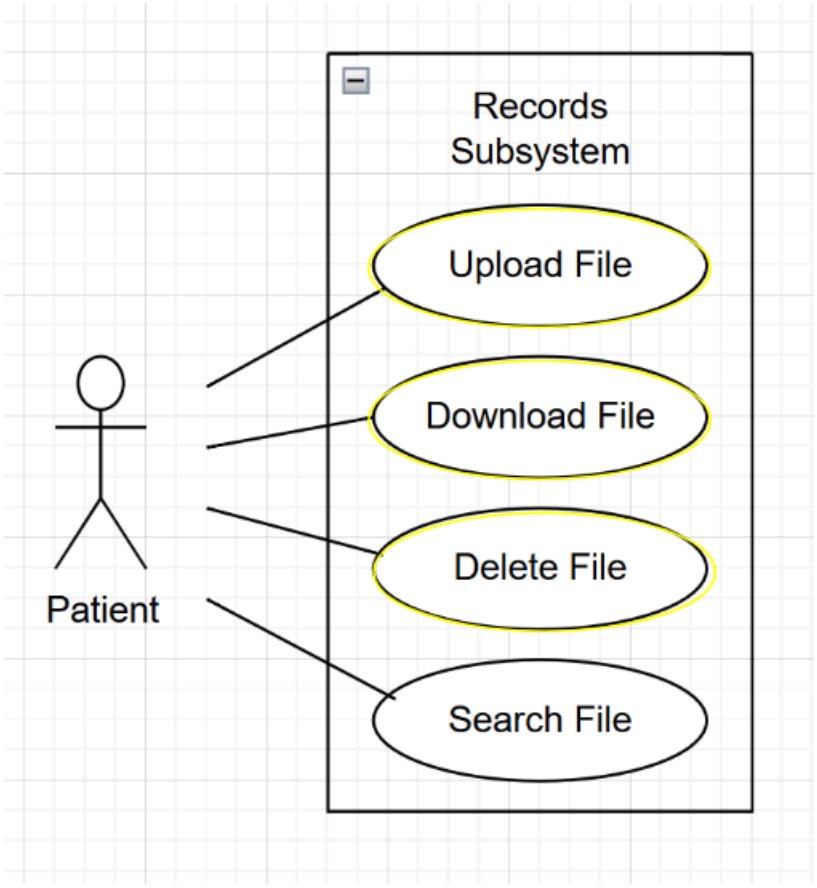
Patient Record Management System developed using Python and Tkinter.

The system allows users to:

- Upload patient files
- Download files
- Delete files
- Search Files

The goal is to provide a simple and organized way to manage patient documents digitally.

**Use Case Diagram:**




**Use Case 1: Upload**

**Description:**

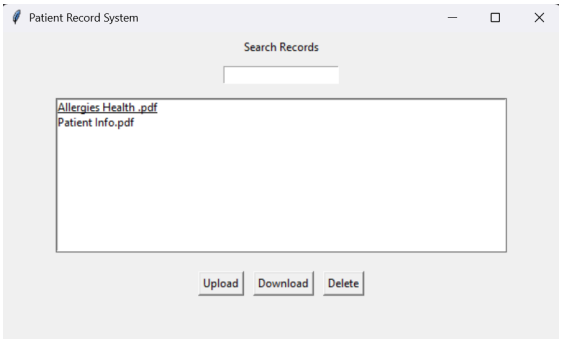
This use case allows the user to upload a medical file into the system. The selected file is copied from its original location into the system's designated storage folder. Once copied, the file list updates automatically to display the newly uploaded record.

**Process:**

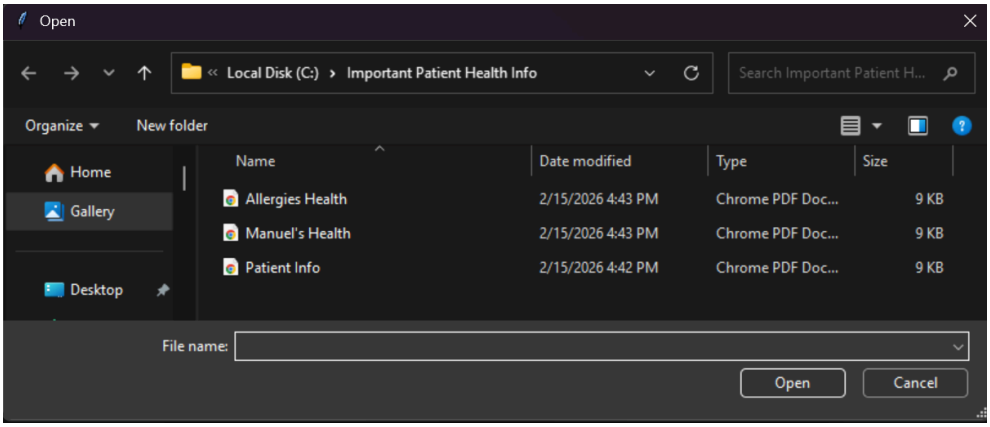
1. User clicks the Upload button.
2. A file selection dialog appears.
3. User selects a file.
4. The file is copied into the record_files directory.

5. The system refreshes the file list.
6. A success message is displayed.

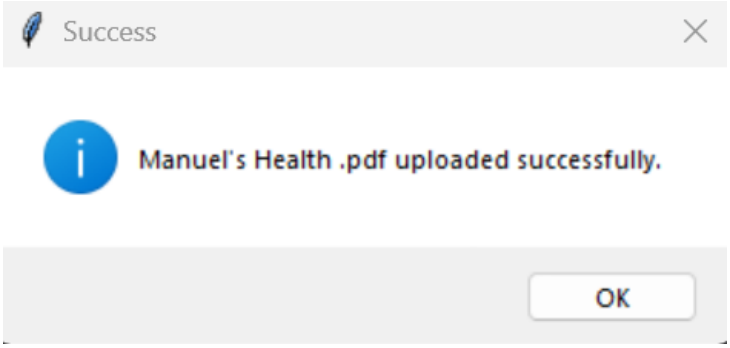
**User Clicks Uploads**




**File Directory is Opened**




**Upload Success message**




**Use Case 2: Download**

**Description:**

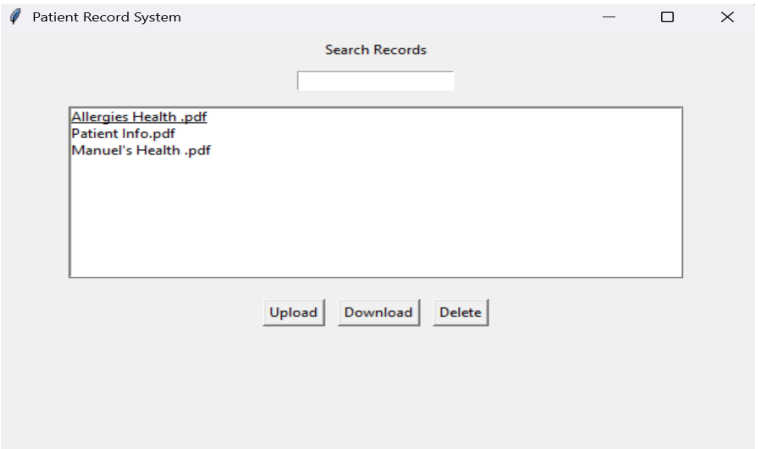
This use case allows the user to retrieve a stored patient file from the system and save it to a selected location on their device.

**Process:**

1. User selects a file from the list.
2. User clicks the Download button.
3. A save location dialog appears.

4. User selects destination folder.
5. The file is copied to the selected location.

**User selects Download**




**File Directory Opens: User selects save**




**Download Success Message**

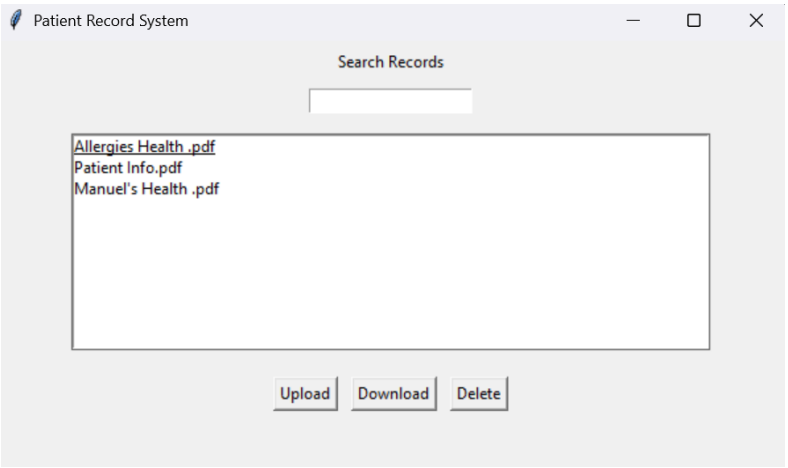



**Use Case 3: Delete**
**Description:**
This use case allows the user to permanently remove a patient file from the system storage.

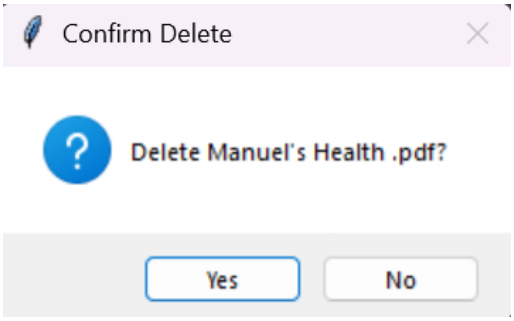
**Process:**

1. User selects a file.
2. User clicks the Delete button.
3. System asks for confirmation.
4. If confirmed, the file is removed from the storage folder.
5. The file list updates automatically.

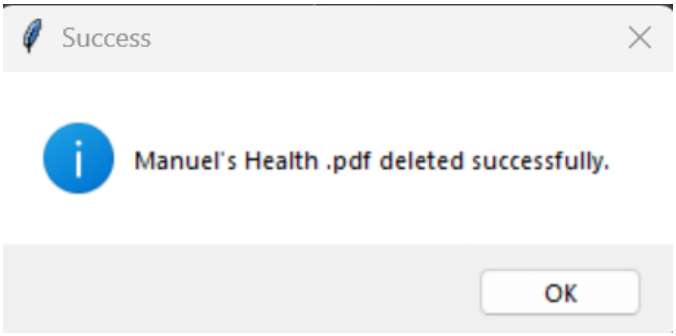
**User Selects Delete**




**Confirmation Message**




**Success Message**




**Database:**

| id | filename | filepath | upload_date |
|---|---|---|---|
| 1 | Patient Info.pdf | record_files\Patient Info.pdf | 2026-02-16 10:34:41 |
| 2 | Manuel's Health .pdf | record_files\Manuel's Health .pdf | 2026-02-16 10:34:47 |
| 3 | Allergies Health .pdf | record_files\Allergies Health .pdf | 2026-02-16 10:34:56 |
| 4 | P1 Health.pdf | record_files\P1 Health.pdf | 2026-02-16 10:43:52 |
| 5 | P2 Health.pdf | record_files\P2 Health.pdf | 2026-02-16 10:44:15 |
| 6 | P3 Health.pdf | record_files\P3 Health.pdf | 2026-02-16 10:44:33 |
| 7 | P4 Health.pdf | record_files\P4 Health.pdf | 2026-02-16 10:44:46 |
| 8 | P5 Health.pdf | record_files\P5 Health.pdf | 2026-02-16 10:44:58 |
| 9 | P6 Health.pdf | record_files\P6 Health.pdf | 2026-02-16 10:45:23 |
| 10 | P67 Health.pdf | record_files\P67 Health.pdf | 2026-02-16 10:45:42 |

**SQL Queries:**

**Table Creation**

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    filename TEXT NOT NULL,
    filepath TEXT NOT NULL,
    upload_date TEXT NOT NULL
)
""")
conn.commit()
```

**Upload**

```
cursor.execute(
    "INSERT INTO records (filename, filepath, upload_date) VALUES (?, ?, ?)",
    (filename, dest_path, upload_date)
)
conn.commit()
```

**Download**

```
cursor.execute("SELECT filepath FROM records WHERE filename = ?", (filename,))
result = cursor.fetchone()
```

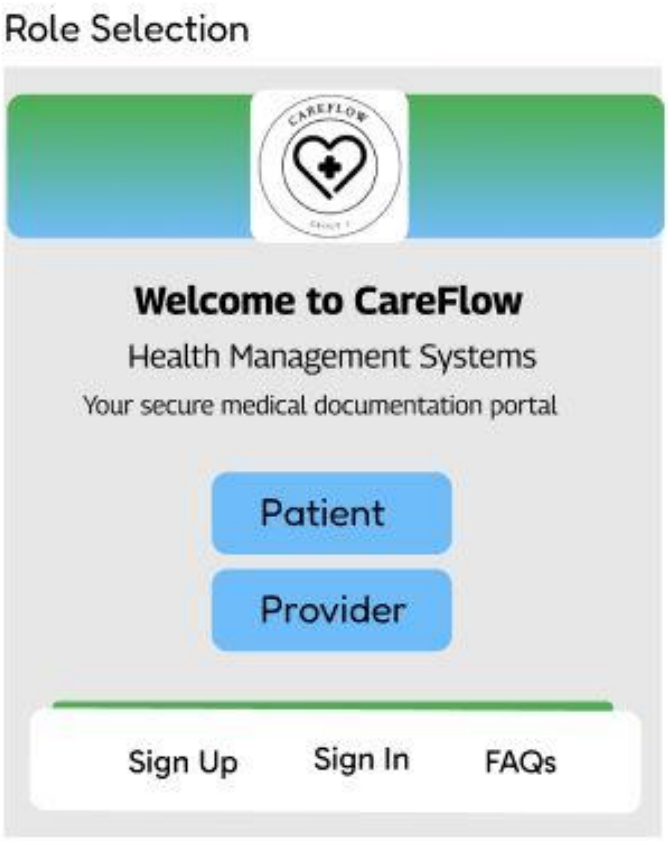
**Delete**

```
cursor.execute("DELETE FROM records WHERE filename = ?", (filename,))
conn.commit()
```

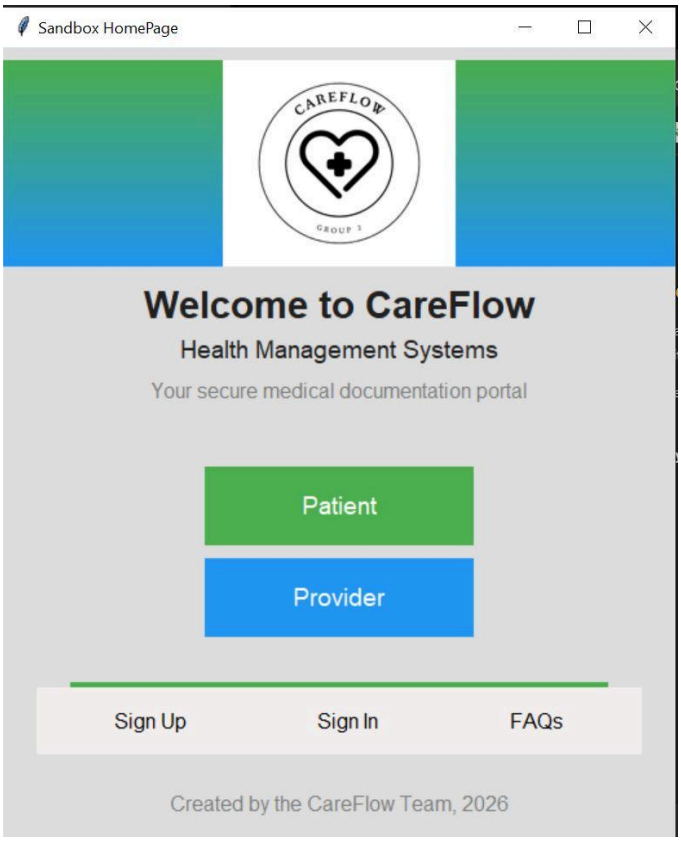
```
SELECT * FROM records;
```

| | id | filename | filepath | upload_date |
|---|---|---|---|---|
| 1 | 1 | Patient Info.pdf | record_files\Patient Info.pdf | 2026-02-16 10:34:41 |
| 2 | 2 | Manuel's Health .pdf | record_files\Manuel's Health .pdf | 2026-02-16 10:34:47 |
| 3 | 3 | Allergies Health .pdf | record_files\Allergies Health .pdf | 2026-02-16 10:34:56 |
| 4 | 4 | P1 Health.pdf | record_files\P1 Health.pdf | 2026-02-16 10:43:52 |
| 5 | 5 | P2 Health.pdf | record_files\P2 Health.pdf | 2026-02-16 10:44:15 |
| 6 | 6 | P3 Health.pdf | record_files\P3 Health.pdf | 2026-02-16 10:44:33 |
| 7 | 7 | P4 Health.pdf | record_files\P4 Health.pdf | 2026-02-16 10:44:46 |
| 8 | 8 | P5 Health.pdf | record_files\P5 Health.pdf | 2026-02-16 10:44:58 |
| 9 | 9 | P6 Health.pdf | record_files\P6 Health.pdf | 2026-02-16 10:45:23 |
| 10 | 10 | P67 Health.pdf | record_files\P67 Health.pdf | 2026-02-16 10:45:42 |

## GUI Development



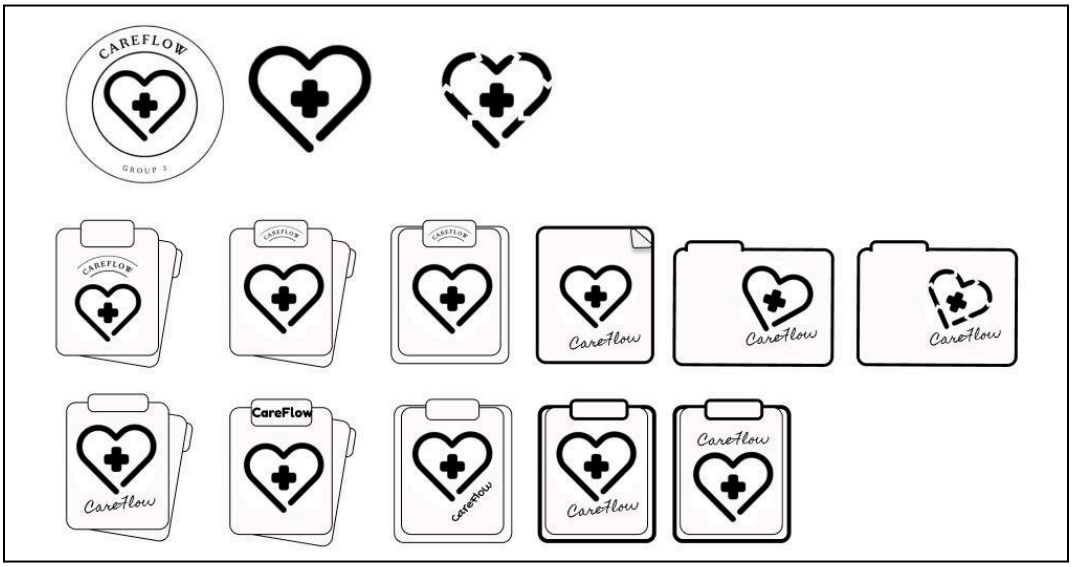

**Role Selection Figma Mockup**

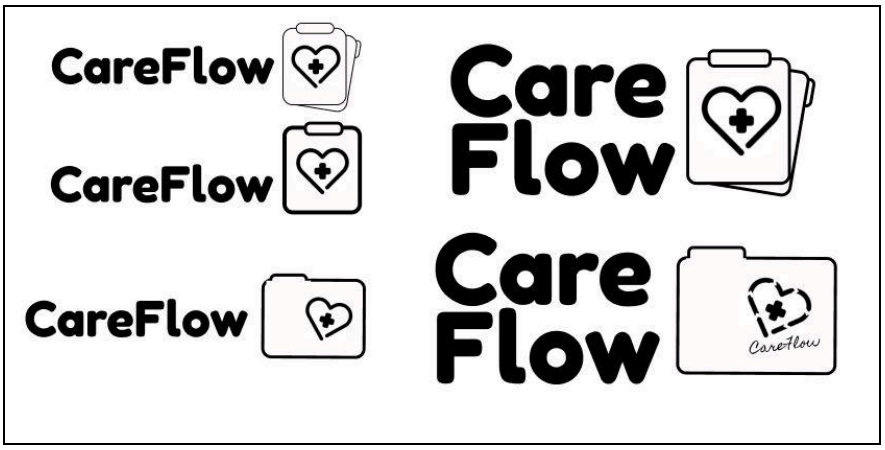



**Role Selection GUI - current**

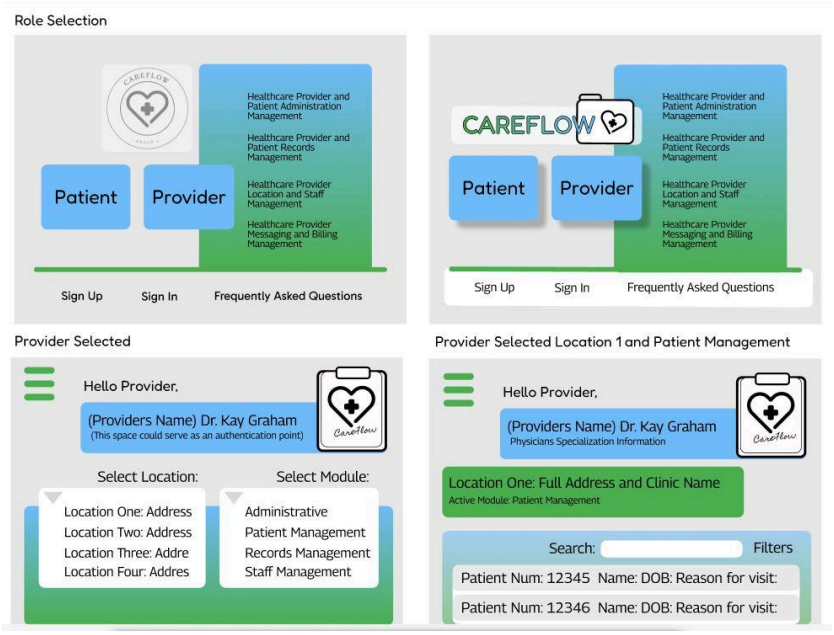
# End of PR2

**PR1: Logo Development Ideation (Tessa)**

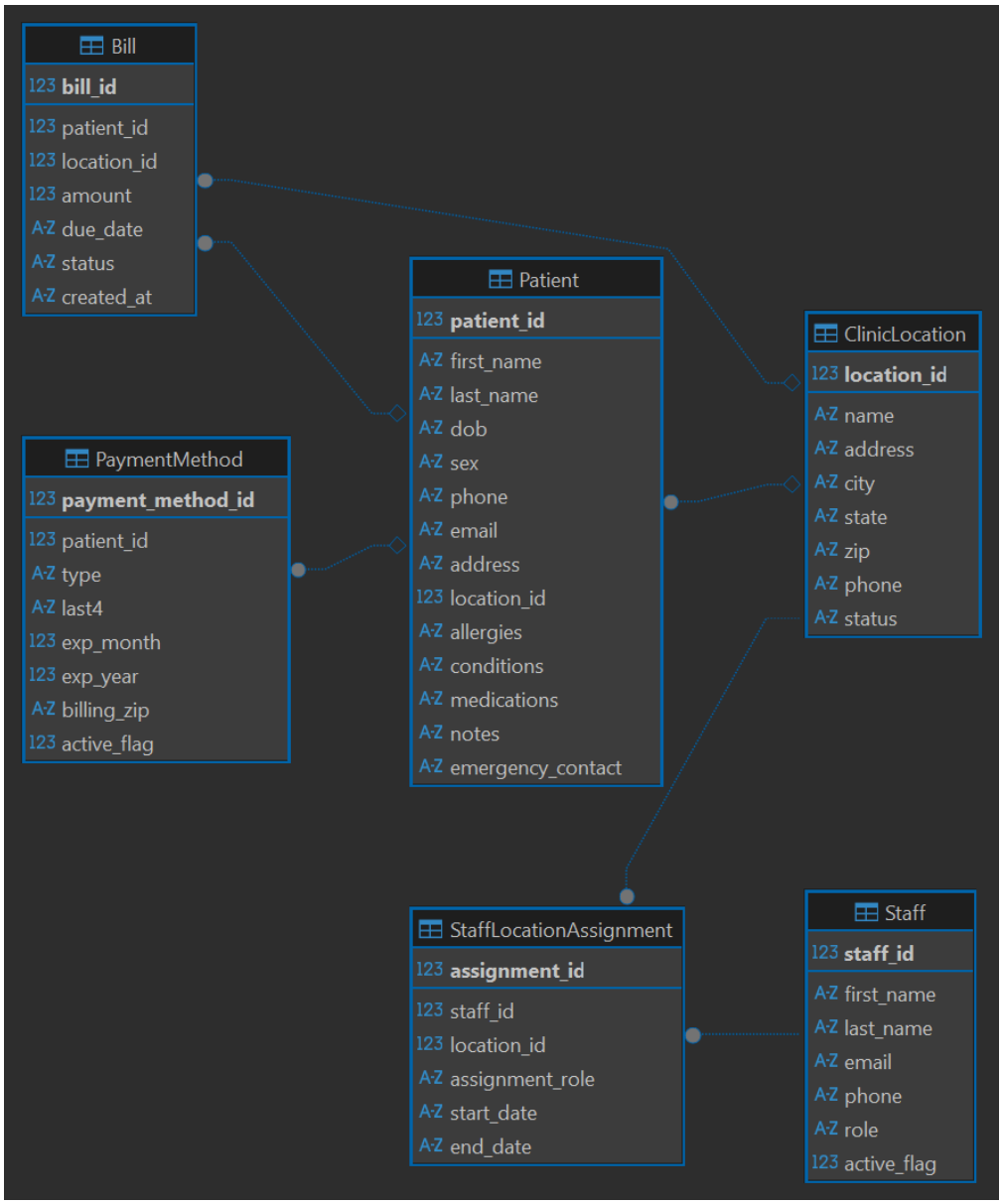




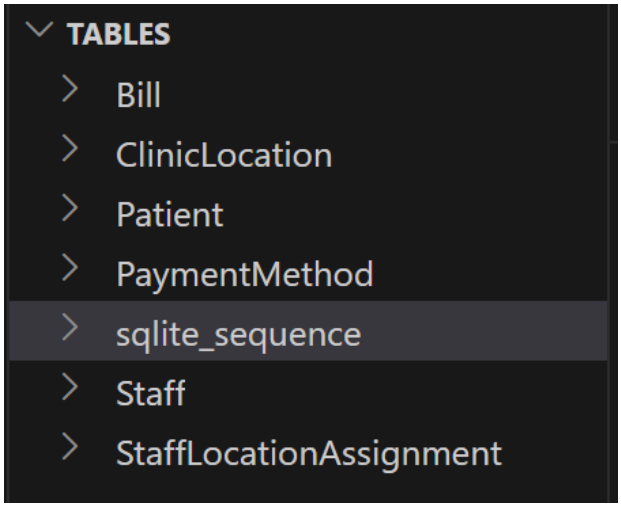

## Interface Development / Wireframe Part 1 (Tessa)




## ERD Updated




**Database Tables SQLite (Logan Templin)**




**Bill:**

| bill_id | patient_id | location_id | amount | due_date | status | created_at |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 120.5 | 2026-03-01 | unpaid | 2026-02-11 17:09:30 |
| 2 | 2 | 2 | 85 | 2026-03-05 | paid | 2026-02-11 17:09:30 |
| 3 | 3 | 3 | 200.75 | 2026-03-10 | unpaid | 2026-02-11 17:09:30 |
| 4 | 4 | 4 | 45.25 | 2026-03-12 | paid | 2026-02-11 17:09:30 |
| 5 | 5 | 5 | 310 | 2026-03-15 | unpaid | 2026-02-11 17:09:30 |
| 6 | 6 | 6 | 95.6 | 2026-03-18 | unpaid | 2026-02-11 17:09:30 |
| 7 | 7 | 7 | 150 | 2026-03-20 | paid | 2026-02-11 17:09:30 |
| 8 | 8 | 8 | 60.4 | 2026-03-22 | unpaid | 2026-02-11 17:09:30 |
| 9 | 9 | 9 | 500 | 2026-03-25 | unpaid | 2026-02-11 17:09:30 |
| 10 | 10 | 10 | 75.99 | 2026-03-28 | paid | 2026-02-11 17:09:30 |

**ClinicLocation:**

| location_id | name | address | city | state | zip | phone | status |
|---|---|---|---|---|---|---|---|
| 1 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 2 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 3 | Main Clinic | 123 Health St | Detroit | MI | 48201 | 555-1111 | active |
| 4 | West Clinic | 456 Care Ave | Detroit | MI | 48202 | 555-2222 | active |
| 5 | Northside Clinic | 111 North Rd | Detroit | MI | 48203 | 555-3001 | active |
| 6 | South Health Center | 222 South Blvd | Detroit | MI | 48204 | 555-3002 | active |
| 7 | East Medical | 333 East Ave | Detroit | MI | 48205 | 555-3003 | active |
| 8 | West Care Center | 444 West Dr | Detroit | MI | 48206 | 555-3004 | active |
| 9 | Lakeside Clinic | 555 Lake Shore | Detroit | MI | 48207 | 555-3005 | active |
| 10 | Downtown Medical | 666 Center St | Detroit | MI | 48226 | 555-3006 | active |

**Patient:**

| patient_id | first_name | last_name | dob | sex | phone | email | address | locati... |
|---|---|---|---|---|---|---|---|---|
| 1 | John | Doe | 1990-05-10 | M | 555-1111 | john.doe@email.com | 101 Main St | 1 |
| 2 | Jane | Smith | 1985-03-22 | F | 555-2222 | jane.smith@email.com | 202 Oak Ave | 1 |
| 3 | Michael | Brown | 1978-11-02 | M | 555-3333 | mike.b@email.com | 303 Pine Rd | 1 |
| 4 | Emily | Davis | 1995-07-18 | F | 555-4444 | emily.d@email.com | 404 Cedar St | 1 |
| 5 | Daniel | Wilson | 1988-01-30 | M | 555-5555 | dan.w@email.com | 505 Birch Ln | 1 |
| 6 | Olivia | Taylor | 2001-09-12 | F | 555-6666 | olivia.t@email.com | 606 Maple Dr | 1 |
| 7 | James | Anderson | 1969-04-25 | M | 555-7777 | j.anderson@email.com | 707 Lake Rd | 1 |
| 8 | Sophia | Thomas | 1999-12-15 | F | 555-8888 | sophia.t@email.com | 808 River St | 1 |
| 9 | William | Jackson | 1975-06-08 | M | 555-9999 | will.j@email.com | 909 Hill Ave | 1 |
| 10 | Ava | White | 1993-02-27 | F | 555-0000 | ava.w@email.com | 1000 Elm St | 1 |

| allergies | conditions | medicati... | notes | emergency_contact |
|---|---|---|---|---|
| Peanuts | Asthma | Albuterol | Prefers morning visits | Jane Doe 555-9991 |
| Penicillin | Diabetes | Metformin | Needs wheelchair access | Tom Smith 555-9992 |
| None | High blood pressure | Lisinopril | Monitor BP | Sarah Brown 555-9993 |
| Shellfish | None | None | First-time patient | Mark Davis 555-9994 |
| Latex | Eczema | Hydrocortisone | Skin sensitivity | Laura Wilson 555-9995 |
| None | Migraines | Ibuprofen | Sensitive to light | Emma Taylor 555-9996 |
| Dust | Arthritis | Naproxen | Chronic joint pain | Linda Anderson 555-9997 |
| Gluten | Celiac disease | Dietary control | Strict diet | Anna Thomas 555-9998 |
| Bee stings | None | EpiPen | Carries EpiPen | Karen Jackson 555-9999 |
| Cats | Anxiety | Sertraline | Prefers telehealth | Chris White 555-1000 |

**PaymentMethod:**

| payment_method_id | patient_id | type | last4 | exp_mon... | exp_year | billing_zip | active_flag |
|---|---|---|---|---|---|---|---|
| 1 | 1 | card | 1234 | 5 | 2027 | 48201 | 1 |
| 2 | 2 | card | 2345 | 6 | 2026 | 48202 | 1 |
| 3 | 3 | bank | 3456 | 7 | 2028 | 48203 | 1 |
| 4 | 4 | card | 4567 | 8 | 2027 | 48204 | 1 |
| 5 | 5 | card | 5678 | 9 | 2029 | 48205 | 1 |
| 6 | 6 | bank | 6789 | 10 | 2026 | 48206 | 1 |
| 7 | 7 | card | 7890 | 11 | 2027 | 48207 | 1 |
| 8 | 8 | card | 8901 | 12 | 2028 | 48226 | 1 |
| 9 | 9 | bank | 9012 | 1 | 2029 | 48210 | 1 |
| 10 | 10 | card | 0123 | 2 | 2027 | 48211 | 1 |

**Staff:**

| staff_id | first_name | last_name | email | phone | role | active_flag |
|---|---|---|---|---|---|---|
| 1 | Alice | Brown | alice@clinic.com | 555-5555 | admin | 1 |
| 2 | Bob | Green | bob@clinic.com | 555-6666 | billing | 1 |
| 3 | Carol | White | carol@clinic.com | 555-7777 | records | 1 |
| 4 | David | Lee | david@clinic.com | 555-8888 | patients | 1 |
| 5 | Emma | Taylor | emma@clinic.com | 555-9999 | billing | 1 |
| 6 | Frank | Harris | frank@clinic.com | 555-1010 | records | 1 |
| 7 | Grace | Martin | grace@clinic.com | 555-2020 | patients | 1 |
| 8 | Henry | Clark | henry@clinic.com | 555-3030 | admin | 1 |
| 9 | Ivy | Walker | ivy@clinic.com | 555-4040 | billing | 1 |
| 10 | Jack | Hall | jack@clinic.com | 555-5050 | records | 1 |

**Staff LocationAssignment:**

| assignment_id | staff_id | location_id | assignment_role | start_date | end_date |
|---|---|---|---|---|---|
| 1 | 1 | 1 | Administrator | 2024-01-01 | NULL |
| 2 | 2 | 2 | Billing Manager | 2024-02-01 | NULL |
| 3 | 3 | 3 | Records Clerk | 2024-03-01 | NULL |
| 4 | 4 | 4 | Patient Coordinator | 2024-04-01 | NULL |
| 5 | 5 | 5 | Billing Specialist | 2024-05-01 | NULL |
| 6 | 6 | 6 | Front Desk | 2024-06-01 | NULL |
| 7 | 7 | 7 | Admin Assistant | 2024-07-01 | NULL |
| 8 | 8 | 8 | Records Manager | 2024-08-01 | NULL |
| 9 | 9 | 9 | Patient Support | 2024-09-01 | NULL |
| 10 | 10 | 10 | Clinic Manager | 2024-10-01 | NULL |